Transportation Research Part E 203 (2025) 104366

Contents lists available at ScienceDirect

Transportation Research Part E

journal homepage: www.elsevier.com/locate/tre

A neighborhood-based matheuristic for the vehicle routing problem with delivery options

Kai Yang [a], Zhenzhen Zhang [a,*], Jingyi Zhao [b], Zhe Liang [a]

[a] School of Economics and Management, Tongji University, Shanghai, China
[b] Shenzhen Research Institute of Big Data, Shenzhen 518172, China

ARTICLE INFO

Keywords:
Vehicle routing
Last mile delivery
Delivery options
Matheuristic
Column generation

ABSTRACT

This paper investigates a realistic version of the vehicle routing problem with delivery options (VRPDO) in the context of last-mile logistics management. The problem aims to optimize vehicle routing while considering alternative customer (request) delivery options and service levels for the last-mile delivery. The shared delivery locations can serve multiple requests simultaneously, but the chosen locations should satisfy the specific capacity and service levels, which requires synchronization between different routes. This synchronization of resources makes route planning more consistent with the needs of actual logistics services but greatly increases the complexity of algorithm design. To address this complex problem, a novel neighborhood-based matheuristic is proposed to intensively search for high-quality solutions from both primal and dual perspectives. Then, a relax-and-fix based insertion heuristic is developed to obtain initial feasible solutions for the proposed algorithm, which effectively balances the feasibility and optimality in solution construction. Finally, the performance of the proposed algorithm is evaluated on two sets of VRPDO benchmarks (small and large instances) available in the literature. The experimental results show that our proposed algorithm has superior performance compared to state-of-the-art methods, obtaining 114 out of 120 and 85 out of 120 best-known solutions for the small- and large-scale benchmark instances, respectively. Furthermore, a real VRPDO case study of a third-party logistics company in China is introduced, which confirms that the proposed algorithm is suitable for processing VRPDO in newly introduced real-world instances.

1. Introduction

The motivation of this paper stems from a real-world operational challenge faced by a third-party logistics (3PL) company operating in China. The company specializes in providing personalized last-mile logistics services to its customers, who can place orders each with multiple delivery locations, including individual delivery locations (IDL) and shared delivery locations (SDL), each of which has varying priorities. For example, some customers order seafood but are at work during the day, so they hope the company can prioritize delivering the seafood at home (IDL) in the evening or alternatively store it at a nearby shop (SDL) to avoid delivery failure or spoilage of the seafood. These alternative delivery locations with specified preferences are referred to as *delivery options*, which are provided to the 3PL company by customers before the official execution of the route planning. The 3PL company strives to fulfill these requests by delivering as much as possible to the customer's desired option while minimizing the total delivery

* Corresponding author.
E-mail addresses: yangkai0119@126.com (K. Yang), zhenzhenzhang222@gmail.com (Z. Zhang), jingyi.z@u.nus.edu (J. Zhao), liangzhe@tongji.edu.cn (Z. Liang).

https://doi.org/10.1016/j.tre.2025.104366
Received 9 February 2025; Received in revised form 25 July 2025; Accepted 2 August 2025
Available online 25 August 2025
1366-5545/© 2025 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

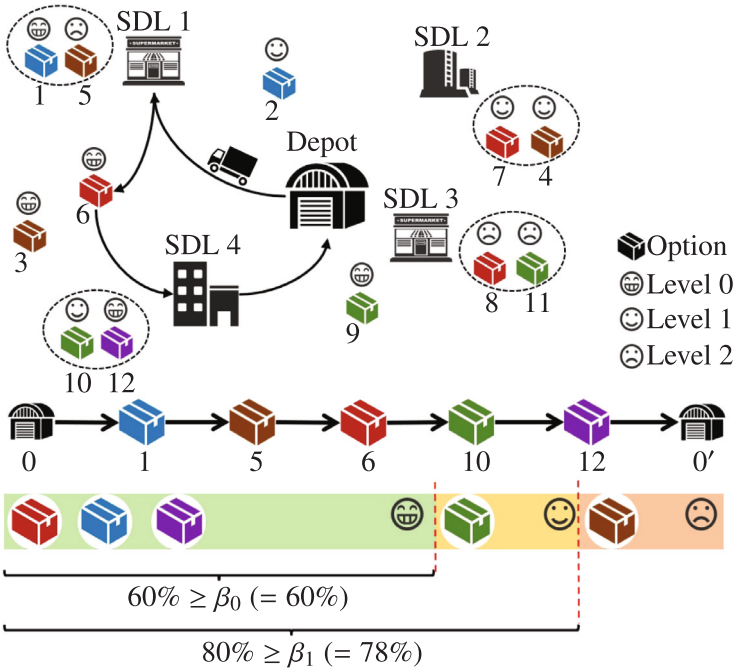


Fig. 1. Example of a VRPDO instance: this system consists of 12 delivery options for 5 requests, where options inside the circle are served by SDL, options outside the circle are served by IDL, and options for each request are represented in the same color. Service level for this case are predefined to ensure that at least 60% of requests receive their preferred primary option (β_0), and that at least 78% of requests are offered either the primary or the next level option (β_1). *Source:* Created by the authors.

cost. With the rapid growth of e-commerce, the demand for orders has skyrocketed, placing tremendous pressure on 3PL to maintain a high standard of the personalized logistics service within its limited budget and manpower. In practice, the 3PL company is often forced to make real-time adjustments to many of its delivery services, rendering the original route planning program obsolete. This situation has severely hampered 3PL's efforts to improve customer service levels.

To address this challenge, the problem is formulated as the vehicle routing problem with delivery options (VRPDO), which was introduced by Tilk et al. (2021) and Dumez et al. (2021). The objective of the VRPDO is to determine a set of min-cost routes that consider alternative delivery locations. In addition to the traditional time windows and vehicle capacity constraints, some other practical factors, such as SDL capacity and service level constraints, are also taken into account. Fig. 1 shows a simple example of the VRPDO with a single vehicle, where 5 requests are represented by different colors. Each request typically has several available delivery options, and each option is prioritized differently. For example, options 6, 7, and 8 belong to the same request and indicate the delivery to IDL, SDL 2, and SDL 3, respectively. At the same time, an SDL can usually serve different requests, e.g., options 1 and 5 are served by SDL 1, while an IDL can only serve a specific option. A vehicle route is represented by an ordered list of option nodes $(0, 1, 5, 6, 10, 12, 0')$, where 0 and $0'$ represent the origin and destination depots, respectively. In addition, the service level in the VRPDO formalizes customer satisfaction through nested structures. Specifically, each delivery option has a priority $p \in \{0, 1, 2\}$, where a lower p indicates a higher customer preference. A feasible solution must guarantee that the proportion of requests served through options of priority p (or lower p) does not fall below the prescribed threshold β_p.

In fact, Tilk et al. (2021) and Dumez et al. (2021) proposed solution approaches for VRPDO. Specifically, Tilk et al. (2021) proposed a branch-and-price-and-cut (BPC) algorithm, which can obtain optimal solutions, but is limited to small-scale instances with 25 or 50 requests and requires a long runtime of 3600 s. In contrast, Dumez et al. (2021) proposed a large neighborhood search (LNS) algorithm, which can solve instances with up to 400 requests in 2000 s. However, this approach struggles to balance solution quality and computational efficiency. Moreover, neither (Tilk et al., 2021) nor (Dumez et al., 2021) provided the construction methods for initial feasible solutions, which restricts the practical applicability of their algorithms, particularly in scenarios requiring rapid responses to unforeseen events. Thus, a novel neighborhood-based matheuristic (NMH) algorithm and a relax-and-fix based insertion heuristic are proposed to efficiently address these gaps in the existing literature. In addition, a real-world case study from the mentioned 3PL company is provided to verify the value of the ground application. Formally, the main contributions of this paper are threefold:

(1) Our proposed NMH combines variable neighborhood descent (VND) with heuristic-based column generation. This hybrid approach aims to explore the search space from both the primal and dual perspectives to improve the best solution. Additionally, the segment-based evaluation scheme is extended to accelerate the entire procedure for VRPDO.
(2) A novel construction method, which is named the relax-and-fix based insertion heuristic (RFIH), is designed to obtain a feasible initial solution quickly. This method is tailored to address SDL capacity and service level constraints, whose promising results are demonstrated in experiments.
(3) Extensive computational experiments are conducted to verify the performance of the proposed NMH on VRPDO benchmark instances. Actually, NMH demonstrates impressive performance compared with the two state-of-the-art algorithms including the BPC algorithm proposed by Tilk et al. (2021) and the LNS algorithm proposed by Dumez et al. (2021), which obtains 114 best-known solutions out of 120 instances and 85 best-known solutions out of 120 instances, respectively. Then, a real-world case study is introduced to demonstrate the practical application of our NMH algorithm on VRPDO.

The remainder of this paper is organized as follows: Section 2 reviews the related literature. The problem is then formally described together with an arc-flow model and a set-partitioning model in Section 3. Section 4 presents the proposed algorithm framework. Sections 5 and 6 report experimental data and computational results for benchmark and real-world instances, respectively. Finally, Section 7 concludes this paper with some closing remarks.

2. Related studies

Since the vehicle routing problem (VRP) was proposed by Dantzig and Ramser (1959), its variants have been widely applied to address various last-mile logistics challenges. The VRPDO comes from VRP family, considers alternative delivery options and their preference levels, thereby posing challenges to problem-solving. This section reviews key contributions to the VRPDO in the literature, which aims to contextualize our contribution within existing studies. First, the direct studies of VRPDO that propose state-of-the-art methods to solve VRPDO are reviewed. Second, some studies of the related problems are mentioned, which motivate us to explore more diverse solution approaches. Finally, given that our proposed NMH can be classified as a column generation-based matheuristic, the relevant literature on this approach is also examined.

Tilk et al. (2021) and Dumez et al. (2021) were the pioneers in addressing VRPDO. Tilk et al. (2021) developed a BPC algorithm from the dual perspective, but this method was only effective for small-scale instances with 25 and 50 requests. They proposed the option-based network and location-based network for VRPDO modeling, and tailored a unified labeling algorithm to solve the pricing problem based on these two networks. Their experiments indicate that the main time consumption of the labeling algorithm occurs in the dominance process, with the location-based network generally requiring more time for dominance than the option-based network on average. In fact, they set a maximum runtime of 3600 s and obtained optimal solutions for 97% and 33% of instances with 25 and 50 requests, respectively. Dumez et al. (2021) constructed a 3-index arc-flow model and a set-covering formulation for VRPDO based on option-based network, and proposed a matheuristic from the primal perspective for larger instances with up to 400 requests in the runtime of 2000 s. This approach treated the set-covering formulation as a post-optimization technique of the LNS algorithm to periodically improve the best solution. However, they ignored the diversity of the route pool, which limited the ability of the set-covering formulation to improve the solutions. Furthermore, both studies failed to propose the construction methods for initial feasible solutions, while an effective construction method is crucial for ensuring timely responses in sudden scenarios. In our work, the arc-flow model of Dumez et al. (2021) is extended by introducing 2-index formulation, which is used for the initial solution construction in the NMH framework. Then, the metaheuristic is integrated with the set-partitioning model to effectively enhance the best solution. In order to improve the diversity of the route pool, the dynamic perturbation and restart strategies are constructed for the metaheuristic to explore more neighborhoods, and the column generation is introduced to search from the dual perspective to enrich the route pool.

Although the direct research on VRPDO is limited in literature, related research on vehicle routing with alternative delivery locations provides valuable methodological insights for neighborhood structures. Existing approaches fall into three main categories. First, Reyes et al. (2017) proposed to destroy-recreate neighborhoods with the route optimization procedure for VRP with roaming delivery locations (VRPRDL), in which locations of adjacent requests are changed when inserting or removing a request from a route. These neighborhoods were extended by Yuan et al. (2021) to the swap move and a local search-based columns generation in addressing the generalized VRP with time windows (GVRPTW). Second, recent studies worked on an adaptive large neighborhood search with a general local search to improve the flexibility of the algorithm in neighborhood exploration. For example, Frey et al. (2023) and Janinhoff et al. (2024) introduced these frameworks for VRP with flexible delivery locations and delivery options for multitrip constraints, respectively. Third, VND with tailored neighborhood operators is also very popular in recent studies, such as the option-based operators proposed by Yu and Anh (2025) for electric VRP with covering locations, and the network-specific operators proposed by Kötschau et al. (2025) for mobile locker deliveries. In VRPDO, the change in options can simultaneously affect SDL capacity and service level constraints, potentially causing expensive moves. Therefore, the VND framework is adopted in our work because it is well-suited for finely controlling the local search process in highly constrained solution spaces. On the other hand, tabu search is introduced to improve the efficiency of heuristic-based column generation, which avoids repetitive exploration in the closest neighborhoods, and meanwhile the route optimization procedure of Yuan et al. (2021) is extended for VRPDO in this work to further enrich the diversity of the route pool.

The VRP family typically exhibits a block diagonal structure, and column generation is known as a tailored method to handle this structure. It has a long history as a key component of the branch-and-price framework in efficiently obtaining optimal solutions for VRP and its variants (Costa et al., 2019). Although column generation is often used to obtain relaxation solutions in the branch-and-price framework, it also contributes to improving the best solution. In fact, the column generation records many potential routes in each iteration, and the best combination of these routes often helps to improve the best solution (Baldacci et al., 2008; Wang et al., 2022; Najy et al., 2025). Therefore, some column generation-based heuristic algorithms integrate metaheuristics and column generation, which were used by researchers to solve VRP variants. Parragh and Schmid (2013) proposed a hybrid method for the dial-a-ride problem, combining VND with column generation for solving the subproblem, and they further enhanced the integer solutions using LNS. In the hybrid algorithm proposed by Beheshti and Hejazi (2015) for VRP with a general soft time window, the metaheuristic was used to tackle the subproblems of column generation in integrative phase, and then parallelization of column generation and searching for feasible solutions in collaborative phase. For the column generation-based heuristic proposed by Yuan et al. (2021) to solve the GVRPTW, which was executed in two phases. Initially, a local search algorithm explored the solution space, followed by obtaining routes with negative reduced costs to further refine the best solution. Gao et al. (2023a) proposed a two-phase framework to schedule trucks and drones for cooperative deliveries, in which the first phase used variable neighborhood search to solve the subproblem of column generation, and the second phase tried to obtain the final solution from the collected routes by solving the tailored mixed integer programming (MIP) models. In summary, existing methods are often time-consuming due to heavy reliance on the dual perspective for solution searching, with column generation typically exhibiting slow convergence. In this paper, the column generation is regarded as a diversification mechanism, where the pricing problem is solved heuristically, and the procedure is only triggered when the set-partitioning model fails to improve the best solution. This strategy allows our column generation to achieve a trade-off between time consumption and solution quality in our NMH framework.

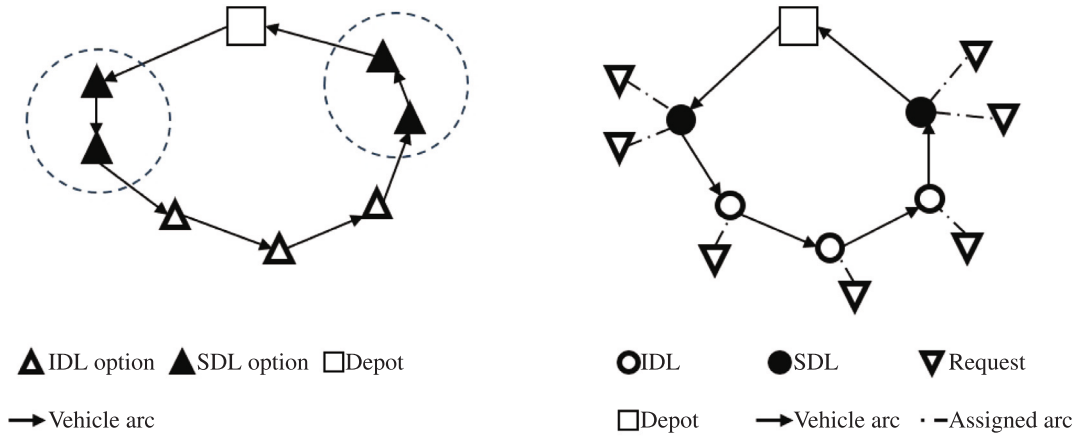
3. Problem description

This section provides a detailed description of VRPDO, which is consistent with the concepts proposed by Tilk et al. (2021) and Dumez et al. (2021). Specifically, each customer request is associated with several delivery options, each defined by a location and a priority level. The VRPDO aims to select exactly one option per request, and construct a set of min-cost routes to serve the chosen options while satisfying vehicle capacity, time window, SDL capacity, and service level constraints. Note that each service level predefines a percentage threshold of requests to be served with options of specific (or higher) customer preference. Mathematically, let $\mathcal{N}$ denotes the set of all requests, $\mathcal{L}$ denotes the set of all locations, $\mathcal{L}_{\text{sdl}}$ denotes the set of all SDL, $\mathcal{P}$ denotes the set of different (delivery) priorities, and $\mathcal{O}$ denotes the set of all delivery options, each of which is a triplet $o = (n_o, l_o, p_o)$ associated with request $n_o \in \mathcal{N}$, location $l_o \in \mathcal{L}$, and priority $p_o \in \mathcal{P}$. For each request $n \in \mathcal{N}$, a demand q_n is given along with several alternative delivery options $\mathcal{O}_n = \{(n_o, l_o, p_o) \in \mathcal{O} \mid n_o = n\}$. Similarly, let $\mathcal{O}_l = \{(n_o, l_o, p_o) \in \mathcal{O} \mid l_o = l\}$ denotes the set of options belonging to location $l \in \mathcal{L}$ and $\mathcal{O}_p = \{(n_o, l_o, p_o) \in \mathcal{O} \mid p_o \le p\}$ denotes the set of options with priority $p \in \mathcal{P}$ or smaller (better). Notably, the starting and ending depots are represented by dummy options o_0 and $o_{0'}$ respectively, but have no priority.

The model of VRPDO is defined over an option-based complete graph $\mathcal{G} = (\mathcal{V}, \mathcal{A})$. As shown in Fig. 2(a), the node set $\mathcal{V}$ is actually the set $\mathcal{O}$, and the arc set $\mathcal{A}$ is defined as $\{(i,j) \mid i \in \mathcal{O} \setminus \{o_{0'}\}, j \in \mathcal{O} \setminus \{o_0\}, i \neq j\}$. Although the option-based graph may involve more candidate nodes than the location-based graph shown in Fig. 2(b), it avoids additional decisions of request-to-location assignment and supports the application of many efficient operators from the classical VRP literature. For parameter description, each arc $(i,j) \in \mathcal{A}$ is associated with a travel cost $c_{i,j}$ and a travel time $t_{i,j}$. It is important to note that the travel time $t_{i,j}$, $(i,j) \in \mathcal{A}$ includes the preparation time, e.g., for parking, at the location of option $j \in \mathcal{O}$ if the two locations are distinct. Each location $i \in \mathcal{L}$ is associated with a service time s_i and a time window $[e_i, f_i]$, and it is assumed that $s_{o_0} = e_{o_0} = e_{o_{0'}} = 0$ and that f_{o_0} and $f_{o_{0'}}$ align with the end of the working day. Every SDL $l \in \mathcal{L}_{\text{sdl}}$ has a capacity D_l, which limits the number of assigned requests. A fleet $\mathcal{K}$ of identical vehicles with capacity Q and fixed cost c_f are available at the depot. Here, the split delivery is allowed for the SDL but forbidden for each request. To guarantee service quality, the percentage of chosen options with a priority less than or equal to p should be at least β_p. Thus, $\beta_p \le \beta_{p+1}, \forall p \in \mathcal{P} \setminus \{|\mathcal{P}|\}$, and $\beta_{|\mathcal{P}|} = 100\%$ always holds, indicating that all requests are consistently fulfilled with at least one assigned option.

3.1. Arc-flow model

In this section, an improved arc-flow model is presented for the VRPDO, which extends the 3-index arc-flow model proposed by Dumez et al. (2021) by introducing the 2-index formulation. This helps reduce the symmetry of the model and has been successfully applied to several VRP variants (Zhang et al., 2019; Pan et al., 2021a). Furthermore, the objective function of our arc-flow model aims to minimize the total cost, encompassing both the fixed cost of vehicles and the travel cost, which is consistent with the objective function of the set-partitioning model proposed by Tilk et al. (2021). This is different from the work of Dumez et al. (2021), which modeled the hierarchical objective of minimizing the number of used vehicles first and then the travel cost. Notably, the objective function with total cost can effectively achieve the same result as the hierarchical objective function if the fixed cost of the vehicle is sufficiently large to dominate the travel cost (Tilk et al., 2021). In general, the use of vehicles can be regarded as fixed cost. For example, in our case study of the 3PL company, vehicles are owned by the company and incur the corresponding maintenance fee and driver salaries for each use, which are usually managed as the fixed cost without dominating



(a) The option-based graph. (b) The location-based graph.

Fig. 2. Visualization of the option-based and location-based graph: the option-based graph (a) focuses solely on routing decisions whereas the location-based graph (b) additionally involves decisions of request-to-location assignment.
Source: Created by the authors.

the travel cost. Thus, it is reasonable to consider the total cost as the objective function for VRPDO modeling. This arc-flow model is mainly used to implement the RFIH method to construct the initial solution in our NMH framework, and more details are described in Section 4.1.

The decision variables are defined as follows:

- y_i $(i \in \mathcal{V})$: binary variable that is set to 1 if node i is visited, and 0 otherwise;
- a_i $(i \in \mathcal{V})$: non-negative continuous variable, which is the service start time of a vehicle at node i;
- d_i $(i \in \mathcal{V})$: non-negative continuous variable, which is the load of a vehicle after serving node i;
- $x_{i,j}$ $((i,j) \in \mathcal{A})$: binary variable, which is equal to 1 if arc (i,j) is traveled by a vehicle, and 0 otherwise.

The arc-flow model for the VRPDO is as follows:

$$\min \sum_{i\in\mathcal{V}} c_f x_{o_0,i} + \sum_{i\in\mathcal{V}}\sum_{j\in\mathcal{V}} c_{i,j} x_{i,j} \tag{1}$$

$$\text{s.t.} \sum_{(i,j)\in\mathcal{A}} x_{i,j} = \sum_{(j,i)\in\mathcal{A}} x_{j,i}, \quad \forall j \in \mathcal{V} \tag{2}$$

$$\sum_{i\in\mathcal{V}} x_{o_0,i} = \sum_{i\in\mathcal{V}} x_{i,o_{0'}} \tag{3}$$

$$a_j \geq a_i + s_i + t_{i,j} + \mathbb{M}(x_{i,j} - 1), \quad \forall (i,j) \in \mathcal{A} \tag{4}$$

$$e_i \leq a_i \leq f_i, \quad \forall i \in \mathcal{V} \tag{5}$$

$$d_i \geq d_j + q_j + (x_{i,j} - 1)Q, \quad \forall (i,j) \in \mathcal{A} \tag{6}$$

$$\sum_{o\in\mathcal{O}_n} y_o = 1, \quad \forall n \in \mathcal{N} \tag{7}$$

$$\sum_{o\in\mathcal{O}_l} y_o \leq D_l, \quad \forall l \in \mathcal{L}_{\text{sdl}} \tag{8}$$

$$\sum_{o\in\mathcal{O}_p} y_o \geq \lceil \beta_p |\mathcal{N}| \rceil, \quad \forall p \in \mathcal{P} \tag{9}$$

$$\sum_{i\in\mathcal{V}} x_{o_0,i} \leq |\mathcal{K}|, \tag{10}$$

$$\sum_{j\in\mathcal{V}} x_{i,j} = y_i, \quad \forall i \in \mathcal{V} \tag{11}$$

$$y_i \in \{0,1\}, \quad \forall i \in \mathcal{V} \tag{12}$$

$$a_i \geq 0, \quad \forall i \in \mathcal{V} \tag{13}$$

$$0 \leq d_i \leq Q, \quad \forall i \in \mathcal{V} \tag{14}$$

$$x_{i,j} \in \{0,1\}, \quad \forall (i,j) \in \mathcal{A} \tag{15}$$

The objective (1) aims to minimize the total cost, including the fixed cost of vehicles and the travel cost. Constraints (2) are the flow conservation constraints, while Constraint (3) guarantees that the vehicle starts from and returns to the depot. To maintain proper scheduling, Constraints (4) ensure the time consistency, where $\mathbb{M} = f_i + s_i + t_{i,j} - e_j$, $\forall (i,j) \in \mathcal{A}$, and Constraints (5) are the time window constraints. Constraints (6) enforce the load consistency for each vehicle. Constraints (7) state that one option is exactly selected for each request. Constraints (8) are the capacity constraints at each SDL $l \in \mathcal{L}_{\text{sdl}}$. Constraints (9) model the satisfaction of service levels. Constraint (10) restricts the number of used vehicles. Constraints (11) establish a link between variables $x_{i,j}$ and y_i, which ensures that if node i is visited, there must exist an arc leaving node i. Lastly, Constraints (12)–(15) define the domain of the decision variables.

3.2. Set-partitioning model

The set-partitioning model has been successfully used by Tilk et al. (2021) to solve the VRPDO, and the same formulation is constructed in our work. Let $\mathcal{R}$ denote the set of all feasible routes. For each route $r \in \mathcal{R}$, c_r denotes the travel cost of route r. The binary coefficient $\alpha_{o,r}$ indicates whether route $r \in \mathcal{R}$ serves option $o \in \mathcal{O}$. The binary variable λ_r decides whether route r is selected in the final solution. Then, the set-partitioning model for the VRPDO is as follows:

$$\min \sum_{r \in \mathcal{R}} (c_f + c_r)\lambda_r \tag{16}$$

$$\text{s.t.} \sum_{r \in \mathcal{R}} \sum_{o \in \mathcal{O}_n} \alpha_{o,r}\lambda_r = 1, \quad \forall n \in \mathcal{N} \tag{17}$$

$$\sum_{r \in \mathcal{R}} \sum_{o \in \mathcal{O}_l} \alpha_{o,r}\lambda_r \leq D_l, \quad \forall l \in \mathcal{L}_{\text{sdl}} \tag{18}$$

$$\sum_{r \in \mathcal{R}} \sum_{o \in \mathcal{O}_p} \alpha_{o,r}\lambda_r \geq \lceil \beta_p |\mathcal{N}| \rceil, \quad \forall p \in \mathcal{P} \tag{19}$$

$$\sum_{r \in \mathcal{R}} \lambda_r \leq |\mathcal{K}| \tag{20}$$

$$\lambda_r \in \{0,1\}, \quad \forall r \in \mathcal{R} \tag{21}$$

The objective (16) aims to minimize the total cost of the chosen routes. Constraints (17)–(20) correspond to Constraints (7)–(10) in the arc-flow model respectively.

4. Solution method

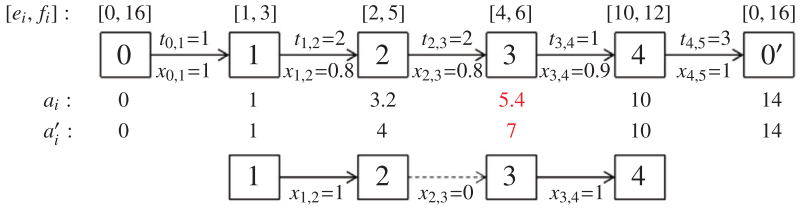
This section outlines the proposed NMH framework, which explores the solution space from both the primal and dual perspectives for finding high-quality solutions. The entire framework is shown in Algorithm 1. First of all, an initial feasible solution is constructed by the RFIH method, and then the route pool is initialized with the initial feasible solution (Lines 1–2). Thereafter, the VND is used to explore the solution space as extensively as possible and generate promising routes into the route pool. Once the size of the route pool exceeds the maximum threshold or the maximum number of adjusting perturbation strength is reached, the VND stops and the set-partitioning model is solved to obtain the best combination of routes in the route pool (Lines 10–28). If the best solution is improved, the VND procedure continues with the new starting point; otherwise, a tabu search-based column generation is employed to expand the route pool from the dual perspective. It is worth noting that the VND is continued with the newly obtained solution after the termination of the column generation regardless of whether the best solution is improved (Lines 29–34). Until the time budget is reached, the NMH is terminated (Lines 5). Formally, the above components of the proposed NMH framework include the RFIH method, the VND framework, the route pool management, and diversification mechanisms, which are detailed in Sections 4.1–4.4. Finally, an acceleration strategy is introduced for solution evaluation in Section 4.5.

4.1. Construction of initial solution

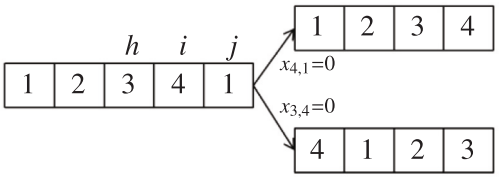
Given the SDL capacity and service level constraints in VRPDO, it is not an easy task to obtain a well-performing initial solution compared to classical VRP. This complexity complicates the trade-off between feasibility and optimality in solution construction. One reason is that excessively pursuing a high service level is likely to cause some SDL to exceed their capacity, especially in extreme cases where the primary option for most requests is SDL. Another reason, from an operational perspective, is that allocating requests to SDL as often as possible helps reduce the total cost, but could lead to a lower service level in scenarios where IDL is the primary option for most requests.

Algorithm 1 The framework of NMH procedure

Construct an initial feasible solution S_0
Initialize route pool $\mathcal{R}$ with S_0
Initial ratio $\eta_1' \leftarrow \eta_1$, $\eta_2' \leftarrow \eta_2$
Best solution $S \leftarrow S_0$, incumbent solution $S' \leftarrow S_0$, temporary solution S''
while the time budget is not reached **do**
  $\Pi_1 \leftarrow 0$, $\Pi_2 \leftarrow 0$
  if $|\mathcal{R}| \geq N_{\max}$ **then**
    $\mathcal{R} \leftarrow$ keep the top $[\delta N_{\max}]$ routes with the least reduced cost
  end if
  while $|\mathcal{R}| < N_{\max}$ and $\Pi_2 < N_2$ **do**
    $S'' \leftarrow$ execute the VND on S' and extend $\mathcal{R}$ with route optimization procedure
    if S'' is not better than S **then**
      if $\Pi_1 < N_1$ **then**
        $S' \leftarrow$ execute the dynamic perturbation on S'', $\Pi_1 \leftarrow \Pi_1 + 1$
      else
        $\eta_1 \leftarrow \zeta\eta_1$, $\eta_2 \leftarrow \zeta\eta_2$, $\Pi_2 \leftarrow \Pi_2 + 1$, $\Pi_1 \leftarrow 0$
        if $\eta_2 < \eta_{\max}$ **then**
          $S' \leftarrow$ execute the dynamic perturbation on S
        else
          $S' \leftarrow$ execute the concat-split method on S, $\eta_1 \leftarrow \eta_1'$, $\eta_2 \leftarrow \eta_2'$
        end if
      end if
    else
      $S \leftarrow S''$, $S' \leftarrow S''$, $\eta_1 \leftarrow \eta_1'$, $\eta_2 \leftarrow \eta_2'$, $\Pi_1 \leftarrow 0$, $\Pi_2 \leftarrow 0$
    end if
    Extend $\mathcal{R}$ with the route optimization procedure on each route $r \in S'$
  end while
  $S' \leftarrow$ best combination of the routes of $\mathcal{R}$ found with a set-partitioning model
  if S' is better than S **then**
    $S \leftarrow S'$
  else
    Extend $\mathcal{R}$ with the tabu search-based column generator and the route optimization procedure
    $S' \leftarrow$ best combination of the routes of $\mathcal{R}$ found with a set-partitioning model
  end if
end while
return The best solution S



(a) The case for time windows, and the service time $s_i = 1,\ i \in \{1, 2, 3, 4\}$.



(b) The case for cycle.

Fig. 3. Examples of two infeasible cases for the generation of route segments.
Source: Created by the authors.

Algorithm 2 The framework of RFIH procedure

Input: Set $\mathcal{N}$ and $\mathcal{V}$
1: Selected options set $\Omega \leftarrow \emptyset$
2: The number of remaining options $u \leftarrow |\mathcal{N}|$
3: **while** $u > 0$ **do**
4: $\Omega \leftarrow$ solve the RAM and fix variables y_i and $x_{i,j}$ whose values are greater than 0.8 to 1
5: **if** $|\mathcal{N}| - |\Omega| = u$ or $|\mathcal{N}| - |\Omega| \leq \lceil \rho |\mathcal{N}| \rceil$ **then**
6: $\Omega \leftarrow$ convert all remaining y_i back to binary variables, and solve the partial MIP model
7: break
8: **end if**
9: $u \leftarrow |\mathcal{N}| - |\Omega|$
10: **end while**
11: Initial solution $S = \{r_1, \ldots, r_k\}$, where r_k $(k \in \mathcal{K})$ is an empty route that starts and ends at depot
12: **while** $|\Omega| \neq 0$ **do**
13: identify the option $o \in \Omega$ and the route-position pair (r, i) such that the resulting route is feasible and the incremental cost is minimized when o is inserted at the position i of route r
14: **if** both option o and position i are found **then**
15: $S \leftarrow$ insert o to route r at position i
16: $\Omega \leftarrow \Omega \setminus \{o\}$
17: **end if**
18: **end while**
19: **return** Initial solution S

In this section, a RFIH method is proposed to construct the initial solution that strikes a balance between feasibility and optimality. The method consists of two phases, in which the first phase aims to iteratively choose the appropriate option for each request, and the second phrase attempts to construct the routes based on the chosen options. The complete procedure is presented in Algorithm 2. In Phase 1 (Lines 3–10), the relaxed arc-flow model (RAM) is defined to treat y_i $(i \in \mathcal{V})$ and $x_{i,j}$ $((i,j) \in \mathcal{A})$ from the arc-flow model as continuous variables. Then, this linear RAM is solved efficiently using a commercial solver. It is noteworthy that there exists a link between y_i $(i \in \mathcal{V})$ and $x_{i,j}$ $((i,j) \in \mathcal{A})$ as expressed in Constraints (11), where fixing $x_{i,j}$ $((i,j) \in \mathcal{A})$ directly leads to the fixation of y_i $(i \in \mathcal{V})$ and y_j $(j \in \mathcal{V})$, thereby aiding in expediting the entire procedure. Therefore, in each iteration, all y_i $(i \in \mathcal{V})$ and $x_{i,j}$ $((i,j) \in \mathcal{A})$ values greater than 0.8 are fixed to 1 and treated as constants in subsequent iterations. This iterative process continues until all y_i $(i \in \mathcal{V})$ variables are fixed.

As more variables $x_{i,j}$ $((i,j) \in \mathcal{A})$ are fixed at 1, certain variables $x_{i,j}$ $(i \in \mathcal{S}, j \in \mathcal{S}, \mathcal{S} \subseteq \mathcal{V} \setminus \{o_0, o_{0'}\})$ can be connected to form a directed flow without the depot, which is called a *route segment*. These route segments must remain feasible to ensure the RAM executes successfully in subsequent iterations. However, as the examples shown in Fig. 3, a route segment is prone to infeasibility in terms of time windows and cycle. For time windows, as shown in node 3 of Fig. 3(a), its relaxed version of the service start time a_3 meets the time window constraint, but the adjusted service start time a'_3 clearly breaks the constraint after the variable $x_{2,3}$ is fixed to 1. Therefore, an effective repair method is to truncate it by fixing the variables $x_{i,j}$ which causes the violation of time windows to 0. For the cycle of option $j \in \mathcal{V}$, as shown in Fig. 3(b), one of the adjacent variables $x_{h,i}$ and $x_{i,j}$ is randomly selected to be fixed at 1, and the other at 0.

Since the influence of Constraints (7), the values of the options corresponding to some requests tend to average out, and none of these variables y_i exceed 0.8 for a long time. Then, the following observation can be obtained:

Observation 1. *As the iteration progresses, the convergence of the relax-and-fix slows down, and can be improved by converting all variables y_i associated with the remaining requests back to binary variables to obtain a partial MIP model and then solve it. The trigger condition is that all y_i corresponding to the remaining requests are less than 0.8 or the remaining number of requests is lower than $\lceil \rho |\mathcal{N}| \rceil$, where ρ represents a predetermined percentage.*

In Phase 2 (Lines 11–18), based on the chosen options in Phase 1, a completely feasible solution is constructed using the best insertion method, which is consistent with the approaches in classical VRP. Its time complexity is $O(|\mathcal{N}|^3)$, since $|\mathcal{N}|$ iterations are required to serve all requests and the time per iteration to identify the best option and position is in $O(|\mathcal{N}|^2)$. Finally, same as the arc-flow model, our solution is represented via options, which is easier for the design of operators in the NMH.

4.2. The VND metaheuristic

In the NMH framework, the VND metaheuristic is used to improve the best solution from the primal perspective. The complete search procedure of the VND is presented in Algorithm 3. In the main loop, the procedure systematically changes the neighborhood structures during the search process, with these structures exploring the search space by performing operators to transition from the current solution to a subsequent one (Line 5). If a better solution is found, the best solution S is replaced by S', and the corresponding operator is recorded in u as the last effective operator (Lines 6–9). The procedure terminates when no further improvement can be achieved with any of the operators. Moreover, to enhance the search effect of VND, an extended search space is considered, which is

allowed to accept specified infeasible solutions. More details on neighborhood structures and search space are described in Sections 4.2.1–4.2.2.

Algorithm 3 The framework of VND procedure

Input: An initial solution S_0
Initial best solution $S \leftarrow S_0$
$u \leftarrow 0$, $i \leftarrow 0$, $flag \leftarrow 0$
while $i \neq u$ or $flag = 0$ **do**
  $flag \leftarrow 1$
  $S' \leftarrow$ apply operator NS_i
  if S' is better than S **then**
    $S \leftarrow S'$, $u \leftarrow i$, $flag \leftarrow 0$
  end if
  $i \leftarrow (i+1) \bmod 4$
end while
return The best solution S

4.2.1. Neighborhood structures

In this work, four operators are proposed that are adaptations of classical operators for application on the option-based graph, and are denoted NS_i ($i \in \{1,2,3,4\}$). In particular, each operator adopts the first-improvement principle to search the neighborhood for a better solution. The details of the four operators are presented below:

- Request-swap (NS_1): Swap the positions of a pair of requests from the same route or different routes, and all possible options of each request are attempted. There are $|\mathcal{N}|^2$ pairs of requests and assume that each request has at most m options, then the time complexity of our NS_1 is $O(m^2 \cdot |\mathcal{N}|^2)$.
- Request-relocate (NS_2): A request is removed, and all its options are considered for relocation to another position in the same route or a different route. The time complexity of classical relocate operator is $O(|\mathcal{N}|^2)$, and our NS_2 considers all possible options for each relocation request. Therefore, the time complexity of NS_2 is $O(m \cdot |\mathcal{N}|^2)$, where m is consistent with that used in NS_1.
- Route-interchange (NS_3): It can be performed within a single route or between a pair of routes. Within a single route (also called 2-opt), a successive segment is reversed; while for two different routes, each of them is divided as two parts and their terminating parts are interchanged. The time complexity of NS_3 is $O(|\mathcal{N}|^2)$, which is consistent with the classical interchange operator.
- Option-optimize (NS_4): This operator is performed on a single route. For a segment of k requests (including the start and end depots), all possible alternative options for these requests are attempted. In this work, k is set to 3. In our implementation, each request is treated as a center point for evaluating the possible option combinations of the segment with its preceding and succeeding adjacent requests. Thus, the time complexity of NS_4 is $O(m^3 \cdot |\mathcal{N}|)$, where m is consistent with that used in NS_1.

4.2.2. Search space

The strict enforcement of SDL capacity and service level constraints significantly limits the search space. Therefore, infeasible solutions are also accepted temporarily in our algorithm, which allows some SDL to exceed their capacity and potentially violate service level constraints with penalties. The modified objective function for a given solution S is as follows:

$$z'(S) = z(S)\left(1 + \varphi\left(\sum_{l \in \mathcal{L}_{\text{sdl}}} P_l(S) + \sum_{p \in \mathcal{P}} P_p(S)\right)\right), \tag{22}$$

where $z'(S)$ and $z(S)$ denote the modified and original objective functions respectively. Moreover, $P_l(S)$ and $P_p(S)$ are the numbers of requests that violate constraints (8) and (9) in the given solution S respectively, and φ is the penalty factor for violations.

More specially, $P_l(S)$ is the number of requests exceeding the constraints (8) in each $l \in \mathcal{L}_{\text{sdl}}$, which can be directly calculated as

$$P_l(S) = \max(0,\ D'_l - D_l), \tag{23}$$

where D'_l denotes the number of requests assigned to the SDL $l \in \mathcal{L}_{\text{sdl}}$ in the given solution S.

For service level constraints, higher service level constraints cover lower service level constraints. Therefore, a hierarchical approach is adopted to count the violation for each service level more accurately. It places emphasis on the minimum number of additional requests required to satisfy the specific service level in a solution S, given that the preceding service level constraints have already been met. Hence, for each priority $p \in \mathcal{P}$, $P_p(S)$ is defined as

$$P_p(S) = \begin{cases} \max\left(0,\ \beta_p|\mathcal{N}| - |\mathcal{N}_p|\right), & if\ p = 0; \\ \max\left(0,\ (\beta_p - \beta_{p-1})|\mathcal{N}| - (|\mathcal{N}_p| - |\mathcal{N}_{p-1}|)\right), & otherwise, \end{cases} \tag{24}$$

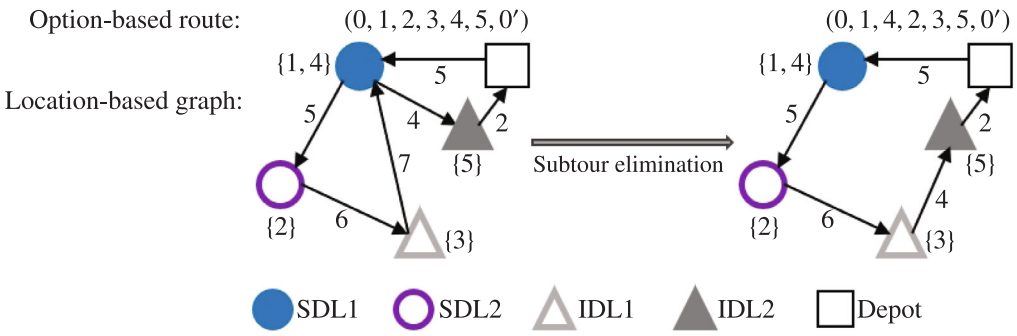


Fig. 4. Example of Observation 2 on the location-based graph: the sets beside the vertices represent the served options by locations, and the numbers beside the arcs are the travel cost.
Source: Created by the authors.

where $|\mathcal{N}_p^*|$ denotes the number of requests in the given solution S that meet priority p.

Since the solution is represented by option-based sequences, an SDL shared in different options could be revisited after serving several physically different locations in the same route, which is called a *subtour*. Then, the following observation can be obtained:

Observation 2. *For an option-based route, the subtour appears frequently, and the corresponding route can be improved via aggregating the options sharing the same SDL by adjusting the visit sequence.*

According to this observation, the route with subtour would not lead to a smaller objective value. At the same time, the corresponding route obtained by eliminating subtours can be easily explored by the operators. Thus, the routes with subtour can be discarded during the search process. Fig. 4 shows an example of Observation 2. Options 1, 2, 3, 4, and 5 are chosen for different requests, in which options 1 and 4 are delivered to SDL1, and options 2, 3 and 5 to SDL2, IDL1 and IDL2, respectively. Obviously, the option-based route $(0, 1, 2, 3, 4, 5, 0')$ is feasible, but it forms a subtour in the location-based graph, which results in a higher travel cost of 29 than the solution obtained by eliminating the subtour with a travel cost of 22.

4.3. Route pool

During the search process of algorithms, numerous temporary solutions are generated, which can be subtly used to guide the search in the later process. These promising routes are recorded in a route pool and used to improve the best solution by solving the set-partitioning model. Notably, the size of the route pool grows dramatically, which makes solving the set-partitioning model computationally demanding. Thus, two strategies are employed to effectively manage the number of routes in the pool.

Firstly, a simple dominance rule is used when adding feasible routes into the route pool. Specifically, each newly generated route is compared with the existing ones serving the same options. If no such route, it is added into the pool directly; otherwise it is added to replace the dominated one if its travel cost is better than the corresponding route in the pool. This is efficiently implemented using a hash table, as shown in Fig. 5, whose key stores the ordered list of options and value records the related information, including the travel cost, reduced cost, option sequence, and locations.

Secondly, the maximum size of the route pool is limited to $N_{\max}$. Once the size of the route pool reaches to $N_{\max}$, a procedure is triggered to retain only the top $\lceil \delta N_{\max} \rceil$ routes with the least reduced cost in the pool, where δ is a user-specific parameter. It is important to note that routes of the current best feasible solution are always preserved to ensure a feasible solution is always available in the route pool. In addition, as the larger number of iterations are performed and more promising routes are explored, the pool size $N_{\max}$ is also dynamically adjusted with an iterative increment strategy. Specifically, $N_{\max}$ is initialized as 10000, and is increased by 10000 after each solution of set-partitioning model, and finally stabilizes at 50000.

4.4. Diversification mechanisms

The VND focuses on exploring nearby neighborhoods, resulting in small differences between generated routes. This not only makes the VND susceptible to getting stuck in local optimum, but also makes it difficult for the set-partitioning model to improve the best solution through the route pool. Therefore, it is necessary to incorporate diversification mechanisms to expand the search scope of VND and enrich the route pool. Three diversification mechanisms are introduced in this work, and the details are provided in Sections 4.4.1–4.4.3.

4.4.1. Dynamic perturbation and restart strategies

The dynamic perturbation drives the solution to a new region when VND is trapped in a local optimum. The operator uses

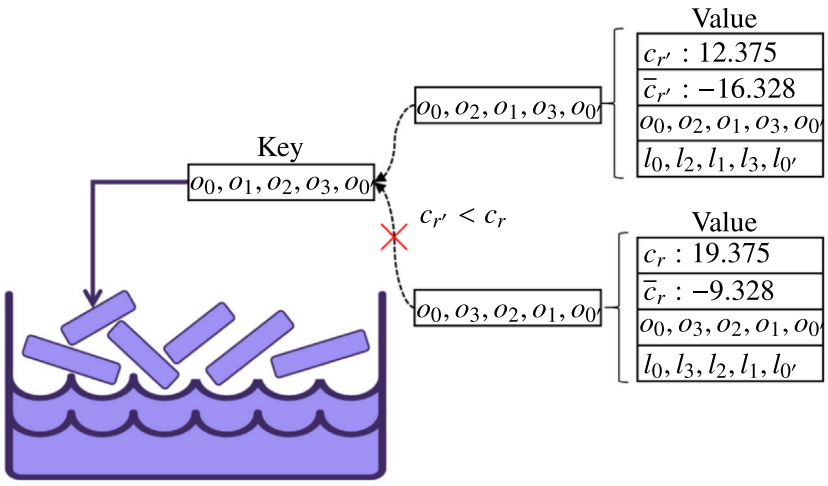


Fig. 5. Example of the route pool and simple dominance rule: the service options (Key) of the new route r' exists in the route pool, and the route pool is updated when the cost of the new route is smaller than the old route r.
Source: Created by the authors.

the ruin-and-recreation principle that is common in the adaptive large neighborhood search and successfully applied to VRP variants (Ropke and Pisinger, 2006; Demir et al., 2012; Christiaens and Vanden Berghe, 2020; Pan et al., 2021b; Zhang et al., 2024). In our implementation, four removal operators are applied, including the worst removal, Shaw removal, historical knowledge node removal, and string removal. These removal operators are the same as those commonly used for VRP variants but just performed on options, and thus maintain the time complexity is the same as that in the literature. For the resulting partial solution, a reg-k insertion method is used for the rest of all options to obtain a new solution, with $k \in \{1, 2, 3\}$ in this work. Assuming each request has at most m options, and $|\mathcal{N}|$ iterations are required to serve all requests and the time per iteration to identify the best option and position is in $O(|\mathcal{N}|^2)$. Moreover, the top k cost increments can be dynamically updated in each iteration, and thus the calculation of the regret value can be easily handled with a time complexity of $O(1)$. Therefore, the time complexity of recreating a solution is $O(m \cdot |\mathcal{N}|^3)$. During each perturbation, a ruin-and-recreation operator pair is selected randomly. Here, two ways are used to obtain a new solution: (1) directly from this operator pair, or (2) by disabling this pair and choosing the best solution from all remaining ruin-and-recreation operator pairs. A parameter $\varrho \in (0, 1)$ is employed to control the choice probability between these two ways.

The number of removed options is dynamically enlarged. Specifically, the number of removed options is defined as $\lceil \eta |\mathcal{N}| \rceil$, where the parameter η is randomly chosen from $[\eta_1, \eta_2]$ with η_1 and η_2 representing the minimum and maximum ratios of options that can be removed respectively. Additionally, a counter Π_1 is used to track the number of iterations that perturbation is performed without improving the best solution. When Π_1 reaches the maximum threshold N_1, both η_1 and η_2 are each enlarged by multiplying by ζ, and the counter Π_1 is reset to 0, and the VND is restarted on the best solution.

Moreover, η_2 is capped at an upper bound $\eta_{\max}$. Once the expansion of η_2 exceeds $\eta_{\max}$, η_1 and η_2 are reset to their respective initial values, and the concat-split method is used as the restart strategy to obtain a new current solution from the best solution. The concat-split method, proposed by Wei et al. (2014), aims to increase the diversity of the search process for VRP variants. Our implementation follows the same method of Wei et al. (2014), but with a key difference: for a given giant tour, five pairs of options are randomly selected to perform 2-opt operators to enhance diversity. It is worth noting that another counter Π_2 tracks the number of adjusting perturbation strength, and triggers the termination of further iterations when Π_2 reaches the maximum threshold N_2. Last but not least, the four parameters η_1, η_2, Π_1, and Π_2 revert to their initial values when the best solution is improved, leading to a steady exploration of the new high-quality neighborhood.

4.4.2. Tabu search-based column generator

Column generation is utilized to expand the route pool from the dual perspective, which applies a Dantzig–Wolfe decomposition to the arc-flow model to yield a master problem (the set-partitioning model in Section 3.2) and a pricing problem. It is usually impossible to enumerate all feasible routes in $\mathcal{R}$, which leads to the restricted master problem, and the pricing problem is an *elementary shortest path problem with resource constraints* which can be defined over the option-based complete graph $\mathcal{G}$ described in Section 3. The pricing problem is solved to identify routes with negative reduced cost $\bar{c}_r$, defined as

$$\bar{c}_r = c_f + c_r - \sum_{n \in \mathcal{N}} \sum_{o \in \mathcal{O}_n} \alpha_{o,r} \theta_n - \sum_{l \in \mathcal{L}_{\text{sdl}}} \sum_{o \in \mathcal{O}_l} \alpha_{o,r} \xi_l - \sum_{p \in \mathcal{P}} \sum_{o \in \mathcal{O}_p} \alpha_{o,r} \psi_p - \mu, \tag{25}$$

where θ_n, ξ_l, ψ_p, μ are the dual values of constraints (17)–(20), respectively.

It is worth noting that our work is different from the labeling algorithm proposed by Tilk et al. (2021) to solve the pricing problem exactly. Here, a tabu search method is proposed to solve the pricing problem, which effectively trade-offs runtime and

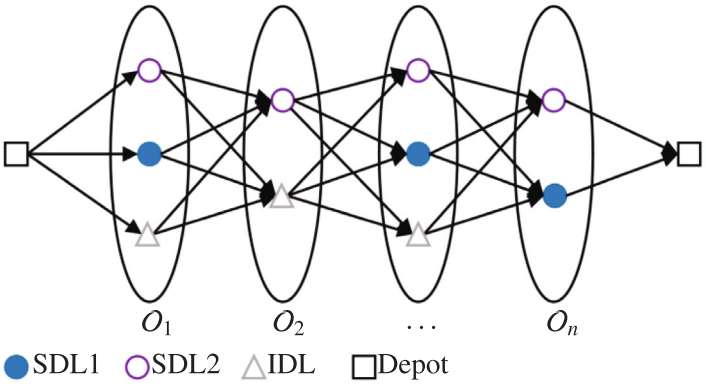


Fig. 6. The layered network of sequence of options set with two SDL and one IDL.
Source: Created by the authors.

solution quality, and has been successfully employed by other researchers for the VRP variants as well (Desaulniers et al., 2006; Archetti et al., 2011; Zhang et al., 2019). Several promising starting points are used for tabu search, which are usually the basic variables of current restricted master problem, and three types of move operators are employed: (1) removing an option from the current route, (2) inserting an option into the current route, and (3) exchanging two options with exactly one option visited by the current route. Without losing generality, the time complexity of the remove operator can be easily derived as $O(|\mathcal{N}|)$. For the insertion operator, there are $|\mathcal{N}|^2$ pairs of position-request and it is assumed that each request has at most m options, thus the time complexity for this operator is $O(m \cdot |\mathcal{N}|^2)$. Similarly, the exchange operator has the time complexity of $O(m \cdot |\mathcal{N}|^2)$, since this operator only considers the alternative options of the requests to be exchanged into the route. To avoid cycling, the tabu list is used to forbid the reverse moves.

The tabu search is invoked for each basic variable and terminated when one of the following criteria is met: (1) no more routes with negative reduced costs are found for three successive iterations, (2) the maximum number of iterations is reached, which is set to 50 in this work. Additionally, the procedure is also terminated once up to 1500 routes with negative reduced costs are identified. Then, at most 150 routes with the lowest reduced costs are added into the route pool.

4.4.3. Route optimization procedure

In VRPDO, each request can have multiple delivery options, leading to various delivery schemes for a given request sequence. To enhance the diversity of the route pool, a route optimization procedure based on the method proposed by Yuan et al. (2021) is introduced. This method is applied when a route is added to the route pool, and it works as follows. Let $\mathcal{N}' \subseteq \mathcal{N}$ be the set of requests to be visited, and given a fixed request sequence $(1, 2, \dots, n)$, $\forall n \in \mathcal{N}'$. It is easy to obtain a sequence of options set $(\mathcal{O}_1, \mathcal{O}_2, \dots, \mathcal{O}_n)$, which is used to find several feasible routes with each serving all these requests. In order to achieve this, a layered network is constructed as depicted in Fig. 6. This network has $n + 2$ layers corresponding to the option sets with their respective vertices, the starting depot, and the ending depot. Then, a labeling algorithm is applied to determine the optimal options for each request in the fixed sequence $(1, 2, \dots, n)$. It is important to note that all feasible routes obtained by the labeling algorithm are attempted to be stored in the route pool, and readers can refer to Yuan et al. (2021) for more details on this labeling algorithm.

4.5. Segment-based evaluation scheme

The VND algorithm is a kind of search-based algorithm in which numerous temporary solutions are generated and evaluated. Therefore, the efficiency of the evaluation method is crucial for the whole performance of the proposed algorithm, especially for the complicated VRPDO problem. To accelerate the feasibility check and cost evaluation of a given solution, Vidal et al. (2013) introduced the classical segment-based evaluation scheme for the VRP and its variants, which can perform the evaluation in constant time regardless of the instance size. The key idea is to consider the removal and insertion operators as means of dividing a route into segments and then reassembling the segments into a new route.

According to the previous Observation 2, the solutions with subtours can be discarded directly. However, the classical segment-based evaluation scheme cannot be directly applied to the VRPDO due to the additional concerns on subtour elimination. Therefore, a subtler check is provided to incorporate subtour elimination for the VRPDO. To maintain consistency with the notation used in Vidal et al. (2013), a segment of node visits is denoted as $\sigma = (\sigma_1, \sigma_2, \dots, \sigma_i)$, where $\sigma_1 \in o_0 \cup \mathcal{V}$, and $\sigma_i \in \mathcal{V} \cup o_{0'}$. The concatenation of two segments $\sigma^1 = (\sigma_1^1, \sigma_2^1, \dots, \sigma_i^1)$ and $\sigma^2 = (\sigma_1^2, \sigma_2^2, \dots, \sigma_j^2)$ is denoted as $\sigma^1 \oplus \sigma^2$. In addition, an extra $L(\sigma)$ is introduced to record the set of locations visited in segment σ, namely $L(\sigma) = \left\{ l_1, l_2, \dots, l_i \right\}$. Then, a new feasibility condition for $\sigma^1 \oplus \sigma^2$ is integrated

Table 1
Parameter tuning with IRACE.

Name	Type	Description	Range	Final value
ρ	Real	A percentage of RFIH iterative adjustment	[0.01,0.3]	0.15
δ	Real	A percentage of the retained route pool size	[0.25,0.75]	0.55
ϱ	Real	A probability parameter about the choice of shake method	[0.2,0.8]	0.63
η_1	Real	Initial minimum ratio of requests can be removed in dynamic perturbation	[0.01,0.1]	0.02
η_2	Real	Initial maximum ratio of requests can be removed in dynamic perturbation	[0.1,0.4]	0.14
$\eta_{\max}$	Real	Maximum ratio threshold of η_2	[0.4,0.9]	0.53
ζ	Real	Adjustment factor for the η_1 and η_2	[1.0,2.0]	1.42
φ	Real	Penalty parameter for the search spaces	[0.01,15]	6.6
N_1	Integer	Maximum number of non-improving iterations allowed for dynamic perturbation	[10,30]	21
N_2	Integer	Maximum number of adjustment of perturbation strength	[3,15]	6

into the classical segment-based evaluation scheme, with its formulation as follows:

$$L(\sigma^1) \cap L(\sigma^2) = \begin{cases} \left\{ l_i^1 \right\}, & if\ l_i^1 = l_1^2; \\ \emptyset, & otherwise, \end{cases} \tag{26}$$

where $L(\sigma^1) = \left\{ l_1^1, l_2^1, \ldots, l_i^1 \right\}$ and $L(\sigma^2) = \left\{ l_1^2, l_2^2, \ldots, l_j^2 \right\}$. This means that either no common location is shared in $L(\sigma^1)$ and $L(\sigma^2)$ or only the linking end nodes share the same SDL, implying that no subtour is generated. Finally, the location information of the new segment $\sigma^1 \oplus \sigma^2$ is updated by:

$$L(\sigma^1 \oplus \sigma^2) = L(\sigma^1) \cup L(\sigma^2), \tag{27}$$

where locations in $L(\sigma^2)$ are sequentially appended to $L(\sigma^1)$. It is worth noting that the feasibility condition (26) only involves the first location l_1 and the last location l_i of $L(\sigma)$. Therefore, the start and end locations corresponding to $L(\sigma)$ are additionally recorded, and $L(\sigma)$ with all the locations visited can be represented as binary values, which allows the union and intersection of $L(\sigma^1)$ and $L(\sigma^2)$ to be computed by bitwise operations, and this enables our extended method to maintain the same O(1) time complexity as the feasibility check of Vidal et al. (2013).

5. Computational experiments

In this section, numerical experiments are conducted to evaluate the performance of our proposed NMH on VRPDO instances. The algorithm is coded in Java, and the Gurobi (version 10.0) is used as the MIP solver in this work. All experiments are executed on a computer equipped with an Intel i9-12900 2.40 GHz CPU. All instances and solutions can be found at https://github.com/YangKai119/VRPDO.

5.1. Parameter settings

A randomly selected set from the benchmark and real-world instances are used as training data, and then the parameters of the NMH are fine-tuned using an automatic algorithm configuration tool called the IRACE package (López-Ibáñez et al., 2016). To control the maximum number of configuration runs allowed, a training budget of 3000 runs is set for our tuning. The parameters are detailed in Table 1, which are initially bounded within reasonable ranges, as determined through extensive pre-experiments, and then the final value of configurations returned by IRACE. The best configuration values of the parameters are used in all subsequent experiments.

5.2. The VRPDO benchmark instances

In order to evaluate the performance of the NMH, the comparative analyses are conducted with two state-of-the-art algorithms: the BPC algorithm proposed by Tilk et al. (2021) and the LNS algorithm proposed by Dumez et al. (2021). The instances of Tilk et al. (2021) are divided into six groups based on the number of options per request and the width of time windows: averaging 1.5 and 2 options per request in the V and U groups, as well as small, medium, and large time windows. This dataset comprises instances with 25 and 50 requests, with ten distinct instances generated for each size category. All these benchmark instances are used for our comparison. Apart from the U and V groups tested by Tilk et al. (2021) and Dumez et al. (2021) introduced a new group named UBC, which expands the capacity of the vehicle and SDL based on the U group. They enlarged the instance size to 50, 100 200 and 400 requests. However, only one type of time window is considered. Moreover, the service levels are set to $\beta_0 = 0.8$ and $\beta_1 = 0.9$ in both benchmark instances, and the instance naming conventions are consistent with the original benchmarks. The comparison of the above two benchmarks is detailed in Sections 5.3–5.4.

5.3. Comparison with BPC algorithm

In this section, a comparative analysis is constructed between the BPC algorithm proposed by Tilk et al. (2021) and the NMH

Table 2
Comparative results for the BPC vs. NMH.

$\|\mathcal{N}\|$	Group	Num	BPC						NMH								
									Avg			Best					
			OPT	BSK	N_{veh}	C_{rou}	Time (s)	N_{veh}	C_{rou}	Time (s)	OPT	BSK	N_{veh}	C_{rou}	Time (s)	Gap	
25	V	Small	10	10	10	3	2616.9	40.9	3	2618.4	5.3	10	10	3	2616.9	2.2	–
		Medium	10	10	10	3	2443.6	489.5	3	2443.6	4.6	10	10	3	2443.6	1.6	–
		Large	10	10	10	3	2114.7	938.3	3	2119.7	6.1	10	10	3	2114.7	3.8	–
		Avg	10	10	10	3	2391.7	489.6	3	2393.9	5.3	10	10	3	2391.7	2.5	–
	U	Small	10	10	10	3	2455.8	23.6	3	2459.4	7.7	10	10	3	2455.8	3.2	–
		Medium	10	9	9	3	2192.3	1089.3	3	2172.8	5.5	9	9	3	2172.4	4.0	−0.91%
		Large	10	9	9	3	2440.6	1595.0	3	2441.7	7.8	9	10	3	2436.2	4.6	−0.18%
		Avg	10	9.3	9.3	3	2362.9	902.6	3	2358.0	7.0	9.3	9.7	3	2354.8	3.9	−0.34%
50	V	Small	10	6	6	5.7	4392.2	4090.9	5.6	4331.0	23.3	4	8	5.6	4318.8	18.8	−1.67%
		Medium	8	4	6	5.4	3722.6	4277.2	5.4	3722.1	25.9	3	9	5.4	3704.8	21.8	−0.48%
		Large	7	2	2	5.7	3816.9	5810.9	5.6	3738.5	23.1	2	10	5.6	3713.1	20.8	−2.72%
		Avg	8.3	4	4.7	5.6	3977.2	4726.3	5.5	3930.5	24.1	3	9	5.5	3912.2	20.5	−1.63%
	U	Small	8	6	6	5.9	3820.5	3225.8	5.8	3746.7	25.8	5	9	5.8	3724.9	21.2	−2.50%
		Medium	9	1	1	5.7	4286.9	6993.6	5.6	4150.0	28.1	0	9	5.6	4100.6	26.2	−4.35%
		Large	5	1	1	5.4	3807.2	5918.7	5.4	3734.2	29.3	1	10	5.4	3719.4	27.1	−2.31%
		Avg	7.3	2.7	2.7	5.6	3971.5	5379.4	5.6	3877.0	27.7	2	9.3	5.6	3848.3	24.8	−3.10%

algorithm. Although the primary advantage of the BPC algorithm lies in improving the dual bounds, this comparative analysis is still necessary as it helps assess the correctness of the NMH implementation and its performance in obtaining high-quality feasible solutions in a short time. Without loss of generality, the NMH is executed 5 runs for each instance, with a maximum runtime of 30 s for 25 and 50 requests, which is consistent with the settings used by Dumez et al. (2021) for the LNS algorithm on the instances with 50 requests. Moreover, the setting of the total cost maintains consistency with the work of Tilk et al. (2021): the travel cost and time are calculated using Euclidean distances, multiplied by 10, and then rounded up, while the fixed cost for each vehicle is set to 100000.

Table 2 displays for each group the number of feasible solutions (Num) identified by the BPC algorithm, along with the number of optimal solutions (OPT) and best-known solutions (BKS) achieved by the BPC and NMH algorithms respectively. Meanwhile, the averages of the number of used vehicles (N_{veh}), travel cost (C_{rou}), and runtime (Time(s)) for the best solutions (Best) obtained by each algorithm and the average solutions (Avg) obtained by 5 runs of NMH algorithm are also reported. The percentage gap (Gap) is utilized to assess the relative efficiency of the two algorithms, calculated as $(C_{NMH} - C_{BPC})/C_{BPC}$, where C_{BPC} and C_{NMH} denote the travel cost of best solutions from the BPC and NMH algorithms, respectively. Here, the symbol "-" indicates that both algorithms solve all instances in the group. Fig. 7 provides a visual representation of the number of OPT and BKS found by both algorithms, categorized by time window widths. For detailed results of each instance, please refer to Tables A.1–A.4 in Appendix A.

The results lead to the following observations. First, the NMH exhibits exceptional performance, which obtains 114 BKS out of all 120 instances, especially for 38 new BKS out of 60 instances with 50 requests. It is worth noting that the number of vehicles of the solutions obtained by NMH on all instances with 25 requests are consistent with the BPC algorithm, and NMH also successfully reduces the number of vehicles for 4 instances with 50 requests. Second, for instances with 25 requests, the NMH fails to find the optimal solutions for only 3 instances within a quite short runtime, and this also confirms its correctness. Notably, NMH obtains the optimal solution for all instances in the V group and updates two BKS for the U group. Third, for the instances with 50 requests, the NMH not only finds 15 OPT certificated by the BPC algorithm but also updates new BKS for all the rest 40 non-optimal instances, leading to a reduction of average travel cost for all groups. Specially, the U group with more options poses a significant challenge for problem-solving. The BPC algorithm fails to find optimal solutions for 22 instances of the U group, including 8 instances with no feasible solutions. In contrast, the NMH performs significantly better on the U group, achieving 28 BKS out of 30 instances, including 22 new BKS and reducing the average travel cost by 3.1%. The NMH also has an impressive performance on average solutions of 5 runs on the U group. It achieves the average travel cost of 3877.0, which is better than the average travel cost of 3971.5 obtained by BPC algorithm. Finally, wider time window also increases the complexity of the instances. The NMH demonstrates remarkable performance in groups with medium and large time windows, obtaining 11 and 17 more BKS respectively for the instances with 50 requests. The most significant travel cost reduction achieved by the NMH is on the U group with medium time windows, whose improvement is up to 4.35%.

5.4. Comparison with LNS algorithm

The performance of NMH on large-scale instances is evaluated through a comparison with the LNS algorithm proposed by Dumez et al. (2021). In their work, a hierarchical objective function is employed to minimize the number of used vehicles first and the travel cost second. Therefore, the fixed cost of the vehicle is set at 100000, which is consistent with Tilk et al. (2021), with the aim of achieving effects similar to the hierarchical objective of Dumez et al. (2021) without losing generality. Moreover, the same calculation as Dumez et al. (2021) is adopted for the travel cost and time, while keeping the same execution of algorithm. Specifically,

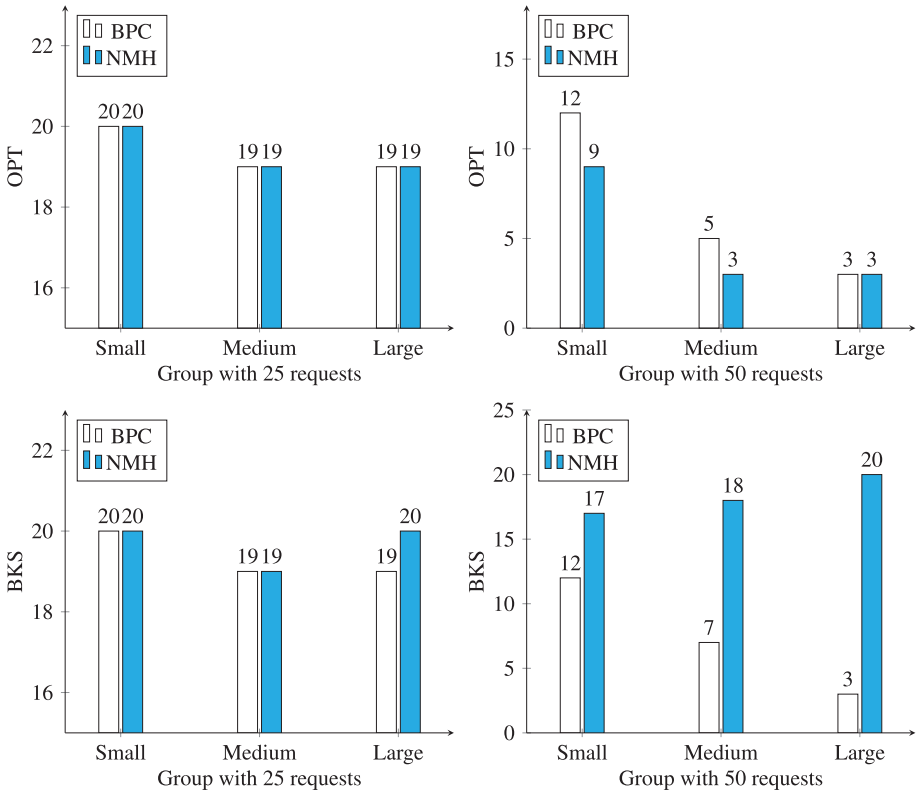


Fig. 7. The number of OPT and BKS are grouped by the time window widths for BPC vs. NMH.
Source: Created by the authors.

Table 3
Comparative results for the LNS vs. NMH.

$\|\mathcal{N}\|$	Group	Avg					Best				
		LNS		NMH			LNS		NMH		
	N_{veh}	C_{rou}	N_{veh}	C_{rou}	Gap	N_{veh}	C_{rou}	N_{veh}	C_{rou}	Gap	
50	U	5.4	391.388	5.4	389.775	−0.41%	5.4	386.448	5.4	387.615	0.30%
	UBC	2	230.132	2	229.510	−0.27%	2	229.353	2	228.276	−0.47%
	V	5.4	375.976	5.4	373.860	−0.56%	5.4	374.259	5.4	371.413	−0.76%
	Avg	4.3	332.499	4.3	331.048	−0.44%	4.3	330.020	4.3	329.102	−0.28%
100	U	10.5	657.774	10.5	655.413	−0.36%	10.5	650.299	10.5	650.842	0.08%
	UBC	4	366.690	4	366.289	−0.11%	4	360.839	4	363.221	0.66%
	V	10.5	717.294	10.4	703.129	−1.97%	10.4	708.906	10.4	698.599	−1.45%
	Avg	8.3	580.586	8.3	574.944	−0.97%	8.3	573.348	8.3	570.887	−0.43%
200	U	20.5	1408.236	20.6	1269.867	−9.83%	20.5	1364.113	20.6	1222.346	−10.39%
	UBC	8	653.472	8	645.680	−1.19%	8	630.259	8	633.760	0.56%
	V	20.5	1526.114	20.7	1444.389	−5.36%	20.5	1478.123	20.7	1415.334	−4.25%
	Avg	16.3	1195.941	16.4	1119.979	−6.35%	16.3	1157.498	16.4	1090.480	−5.79%
400	U	40.6	2567.886	40.6	2479.711	−3.43%	40.6	2514.935	40.6	2467.940	−1.87%
	UBC	15.6	1299.169	15.6	1266.020	−2.55%	15.6	1266.890	15.6	1255.339	−0.91%
	V	40.5	2511.221	40.7	2406.939	−4.15%	40.5	2484.384	40.7	2384.995	−4.00%
	Avg	32.2	2126.092	32.3	2050.890	−3.54%	32.2	2088.736	32.3	2036.091	−2.52%

the travel cost and time are computed as Euclidean distances without any rounding. Then, the NMH is executed 5 runs for each instance, with a maximum runtime of 30 s for 50 requests, 90 s for 100 requests, 300 s for 200 requests, and 2000 s for 400 requests.

Table 3 presents the number of vehicles (N_{veh}) and travel cost (C_{rou}) in both average (Avg) and best (Best) cases. The percentage gap (Gap) is used to evaluate the performance of the NMH, similar to Section 5.3. Additionally, Table 4 counts the number of BKS found by both algorithms for each group in both average and best cases. It also reports the number of new BKS (New) updated by the NMH. Fig. 8 provides a graphical representation of Table 4 for the best cases. The results of each instance are detailed in Tables B.1–B.4 of Appendix B.

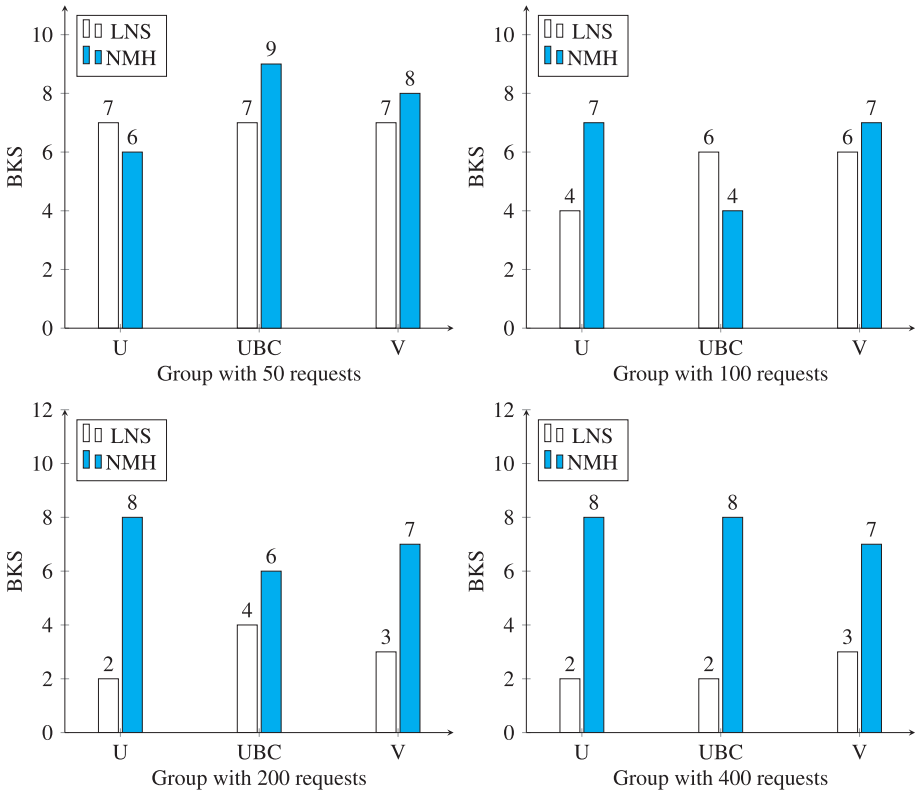


Fig. 8. Bar plot represents: the number of BKS in each group for LNS vs. NMH.
Source: Created by the authors.

Table 4
The number of best known solutions for the LNS vs. NMH.

Group	$\|\mathcal{N}\|$	Num	Avg		Best		
			LNS	NMH	LNS	NMH	New
U	50	10	4	6	7	6	3
	100	10	1	9	4	7	5
	200	10	1	9	2	8	8
	400	10	1	9	2	8	8
	Total	40	7	33	15	29	24
UBC	50	10	6	9	7	9	2
	100	10	3	7	6	4	3
	200	10	0	10	4	6	6
	400	10	3	7	2	8	8
	Total	40	12	33	19	27	19
V	50	10	5	8	7	8	3
	100	10	2	8	6	7	4
	200	10	2	8	3	7	7
	400	10	3	7	3	7	7
	Total	40	12	31	19	29	21

According to the results, it can be observed that the NMH achieves notable results by securing 85 BKS out of 120 instances in the best case, including 64 new BKS, equating to an update rate of 53.33%. Second, the two algorithms use the same number of vehicles for the instances with 50 and 100 requests in the best case, whereas NMH uses an average of 0.1 slightly more vehicles than LNS for the instances with 200 and 400 requests. In fact, the use of more vehicles occurs only in 5 out of all 120 instances. This indicates that our objective function with total cost can achieve a similar effect to the hierarchical objective in the work of Dumez et al. (2021) when the fixed cost is set to 100000. This is due to the efforts of our algorithm to improve the best solution from both primal and dual perspectives. In the primal search, the VND is combined with dynamic perturbation and restart strategies to actively improve the best solution, and add many promising routes for the route pool. An observation is that the necessary randomness introduced by dynamic perturbation and restart strategies might temporarily increase the number of vehicles used. However, these strategies

effectively guide the VND toward high-quality neighborhoods to improve the best solution, thereby enabling the reduction of vehicle usage. On the other hand, column generation significantly expands the route pool with promising routes from a dual perspective, and these routes could be challenging to obtain by VND exploration, but critical for reducing the number of vehicle usage. For example, routes with relatively tight time windows or capacity constraints are typically difficult for the VND to further update. As the diversity of the route pool increases, the ability of set-partitioning model to improve the best solution is simultaneously strengthened, which leads to a greater chance of reducing vehicle usage. Third, our algorithm demonstrates impressive performance and strong stability in travel cost, consistently reducing them across all groups of 50, 100, 200 and 400 requests in the average case. Particularly for instances with 200 requests, the NMH demonstrates outstanding performance in the best case, which results in a significant average reduction of 5.79% in the travel cost across the three groups, notably decreasing by 10.39% and 4.25% for U and V groups respectively. Moreover, NMH reduces the average travel cost across all the three groups in both average and best case for instances with 400 request, which achieves improvements of 3.54% and 2.25%, respectively. These results highlight the superior performance of our algorithm, particularly in handling large-scale instances.

Observed from Table 4, it is evident that the NMH consistently outperforms LNS in both average and best cases, securing more BKS for each group. Furthermore, the NMH performs better on the U and V groups than the UBC group. In the best case, it obtains 14 and 10 more BKS for the U and V groups respectively compared to LNS, whereas it achieves only 8 more BKS for the UBC group. A plausible explanation is that the increased capacities of vehicle and SDL in the UBC group typically allow each vehicle or SDL to serve more requests, which leads to a larger search space and then poses significant challenges for the algorithm in balancing constraints related to time windows and service levels. Nevertheless, the NMH can maintain solid performance in the UBC group, updating a total of 19 BKS, and outperforms LNS across all instances with 200 requests in the average case.

6. Case study: instances from a third-party logistics company

In this section, the proposed NMH is tested on new real-world instances from a third-party logistics company in China. The computational environment and the parameter settings of algorithm are maintained consistency with Section 5. Structurally, a detailed description of the VRPDO real-world instances is provided in Section 6.1, and the primary computational results are presented as the new benchmark for these instances in Section 6.2. Then, the performance of RFIH is investigated in Section 6.3. Thereafter, Section 6.4 investigates the performance of various components for the NMH, and Section 6.5 provides the sensitivity analysis to further evaluate the impact of key parameter settings.

6.1. The VRPDO real-world instances

The dataset comprises daily orders specifying quantities and preferences for delivery options. Each order (request) involves between 1 to 50 items, with up to three alternative delivery options. Service level constraints are predefined to ensure that at least 80% of requests receive their preferred primary option (β_0), and that at least 90% of requests are offered either the primary or the next level option (β_1).

Each delivery option of requests corresponds to a real location categorized into four instances (R1, R2, R3, and R4) based on geographic areas. Detailed information is presented in Table 5, which includes the numbers of requests ($|\mathcal{N}|$), locations ($|\mathcal{L}|$), options ($|\mathcal{O}|$), vehicles ($|\mathcal{K}|$), and SDL ($|\mathcal{L}_{sdl}|$) for each instance (Ins). Fig. 9 shows the geographic locations of the instances mentioned in Table 5, where the SDL are uniformly marked in black and the IDL are color-coded according to their respective geographic areas. In addition, the depot of 3PL company is located within area 1 and is marked with a red triangle in Fig. 9. Notably, these areas are independent, and alternative options for a request are confined within the same instance. Each area has a homogeneous fleet with a capacity of $Q = 600$, which is managed by the depot for a fixed cost of 100 per vehicle. SDL are sourced exclusively from corporate partners and are limited to handling between 5 to 7 requests. The planning horizon for a day spans 13 h (= 780 min), and time windows must be provided within this timeframe. The preparation time at a location is 5 min for IDL and 3 min for SDL. Options at SDL have a service time of 2 minutes, while options at IDL have a service time of 4 min. The vehicle speed is set to 0.67 kilometers per minute, and the distances between any two locations are calculated using the Formula (28), which utilizes real-world latitude and longitude coordinate data, and satisfies the triangle inequality:

$$\omega_{i,j} = \kappa \cdot 12742 \cdot \arcsin\left(\sqrt{\sin^2\left(\frac{\epsilon_i - \epsilon_j}{2}\right) + \cos(\epsilon_i)\cos(\epsilon_j)\sin^2\left(\frac{\iota_i - \iota_j}{2}\right)}\right), \quad \forall (i,j) \in \mathcal{A}, \tag{28}$$

where $\omega_{i,j}$ is the distance between locations i and j, κ is a deviation coefficient which is set to 1.5, and ϵ_i and ι_i are the latitude and longitude for location $i \in \mathcal{L}$ respectively. Formally, the travel cost between locations i and j is calculated as $\varepsilon \cdot \omega_{i,j}$, and ε is a cost factor of per kilometer which is set as 20 in this work.

6.2. Results on VRPDO real-world instances

In this section, the NMH is employed to solve each real-world instance for 5 runs, with a maximum runtime of 300 s per run. This runtime of 300 s comes from the operational experiences of the 3PL company, as they need to make some subtle adjustments to the plan based on certain unpredictability. Thus, the algorithm needs to complete the solution within a short and reasonable runtime. Moreover, our work provides the performance of NMH with longer runtime, which can serve as real-world benchmark for future studies, and these results are detailed in Appendix C.

Table 5
The detailed information of the instances.

Ins	$\lvert\mathcal{N}\rvert$	$\lvert\mathcal{L}\rvert$	$\lvert\mathcal{O}\rvert$	$\lvert\mathcal{K}\rvert$	$\lvert\mathcal{L}_{\text{sdl}}\rvert$
R1	202	219	394	12	30
R2	187	204	357	11	30
R3	96	105	196	6	15
R4	113	121	219	8	15

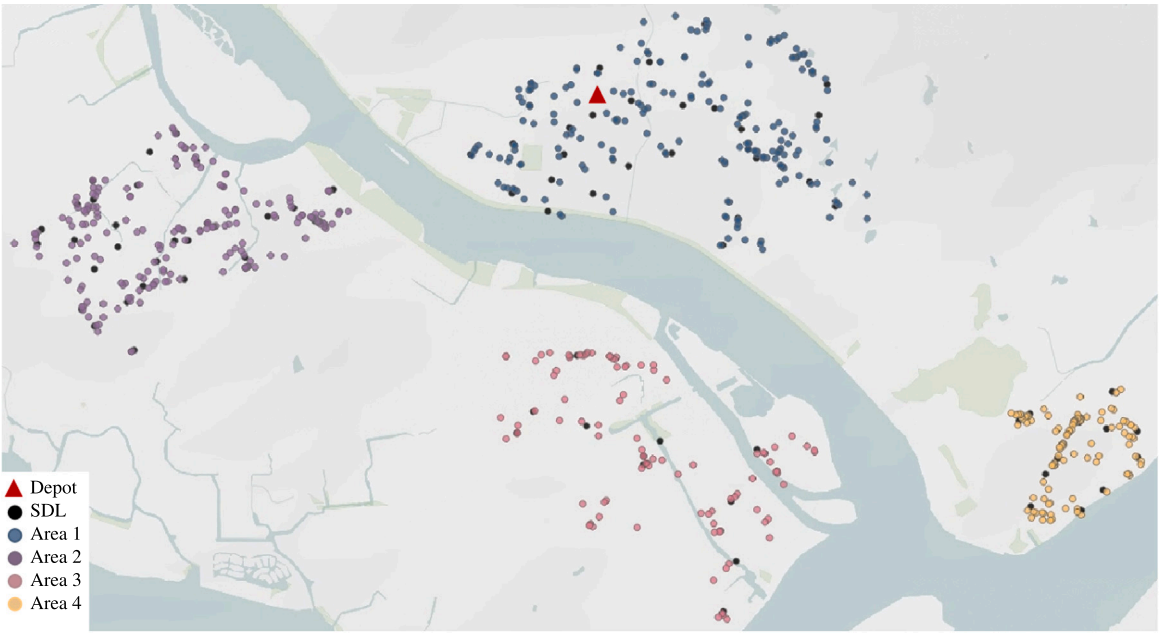


Fig. 9. Geographical display of the 3PL delivery areas on a real map.
Source: Created by the authors.

For comparison, the manual planning (MP) is introduced as the baseline, which is derived from the real-world operations of 3PL company. Table 6 summarizes the results of total cost between the MP and NMH, including the total cost (Z_{tot}), travel cost (C_{rou}), and the number of vehicles used (N_{veh}). In particular, the results of NMH are recorded as average (Avg) and best (Best) cases, respectively, to provide a more comprehensive evaluation of its performance. Additionally, the percentage gap (Gap) between the MP and NMH regarding Z_{tot} is calculated using the same method as in Section 5.3. At the same time, Table 7 presents the utilization of SDL and service levels for MP and NMH. The results of NMH are also recorded as average and best cases. In fact, the utilization of SDL includes two key indicators: the number of used SDL (N_{sdl}) and the utilization rate of SDL capacity (γ). Specifically, the utilization rate of SDL capacity is calculated as $\gamma = \frac{\sum_{l\in\mathcal{L}_{\text{sdl}}} D'_l}{\sum_{l\in\mathcal{L}_{\text{sdl}}} D_l} \times 100\%$, where D'_l denotes the number of requests assigned to the SDL $l \in \mathcal{L}_{\text{sdl}}$. In terms of service levels, Table 7 reports the percentages of requests served by the primary and the next level option in the solutions, which are denoted as β_0^* and β_1^*, respectively.

The results of Table 6 show the excellent performance of NMH in total cost savings. The total cost of NMH is greatly reduced compared to MP in all four instances, and the average total costs of NMH in both the average and best cases are significantly lower than the average total cost of MP, which leads to the reductions of 19.65% and 20.10%, respectively. Especially for the R1 instance, the reductions of total cost in average and best cases are 32.38% and 32.83%, respectively. In fact, it is more challenging to solve R1 instance, because it has the highest number of requests and options which necessitates arranging more services. Additionally, the depot is located within the delivery area, allowing vehicles to approach from all directions and thus leading to more route choices. These reductions in total cost mean that our proposed NMH can effectively provide high-quality solutions in a short time. Moreover, the results of Table 7 provide the insights into the competitiveness of NMH. The MP maintains a slightly higher average service levels, with the primary and the next level option being 82.89% and 96.84% respectively, compared to the average service levels of 80.29% and 95.57% in the best case maintained by NMH. However, the average utilization rates of SDL capacity in MP is 66.27%, which is lower than the 70.54% achieved by NMH in its best case. This indicates that MP tends to prioritize requests with IDL as the primary option in order to improve service levels, which results in higher travel costs and more vehicles. The average travel cost and vehicle count for MP are 1724.88 and 8, while NMH achieves 1342.48 and 6.75 in the best case. These results show that NMH can obtain more cost-effective solutions by better balancing the utilization of SDL and service levels.

Moreover, the results highlight the significant impact of geographic characteristics on both total cost and SDL utilization. A notable observation is that placing the depot within the delivery area reduces total costs. For example, the R1 where the depot

Table 6
Results on the cost information for VRPDO real-world instances.

Ins	MP			Avg				Best			
	Z_{tot}	C_{rou}	N_{veh}	Z_{tot}	C_{rou}	N_{veh}	Gap	Z_{tot}	C_{rou}	N_{veh}	Gap
R1	2787.12	1687.12	11	1884.55	984.55	9	−32.38%	1872.24	972.24	9	−32.83%
R2	3280.88	2280.88	10	2982.28	2082.28	9	−9.10%	2965.02	2065.02	9	−9.63%
R3	1539.90	1039.90	5	1271.84	871.84	4	−17.41%	1263.15	863.15	4	−17.97%
R4	2491.63	1891.63	6	1976.52	1476.52	5	−20.67%	1969.52	1469.52	5	−20.95%
Avg	2524.88	1724.88	8	2028.80	1353.80	6.8	−19.65%	2017.48	1342.48	6.8	−20.10%

Table 7
Results on the utilization of SDL and service levels for VRPDO real-world instances.

Ins	MP				Avg				Best			
	N_{sdl}	γ	β_0^*	β_1^*	N_{sdl}	γ	β_0^*	β_1^*	N_{sdl}	γ	β_0^*	β_1^*
R1	30	67.78%	82.67%	96.04%	30	70.67%	80.20%	96.04%	30	70.00%	80.20%	95.05%
R2	28	59.32%	85.03%	95.19%	28.4	64.18%	80.21%	96.26%	29	64.97%	80.21%	97.33%
R3	14	70.93%	83.33%	97.92%	15	75.81%	80.21%	92.92%	15	75.58%	80.21%	91.67%
R4	14	67.05%	80.53%	98.23%	13.8	71.59%	80.53%	97.70%	14	71.59%	80.53%	98.23%
Avg	21.5	66.27%	82.89%	96.84%	21.8	70.56%	80.29%	95.73%	22	70.54%	80.29%	95.57%

Table 8
Comparative results for the construction of initial feasible solution.

Ins	RFIH			Greedy				Regret			
	Z_{tot}	C_{rou}	N_{veh}	Z_{tot}	C_{rou}	N_{veh}	Gap	Z_{tot}	C_{rou}	N_{veh}	Gap
R1	2365.47	1465.47	9	2753.04	1853.04	9	16.38%	2624.88	1724.88	9	10.97%
R2	3190.51	2290.51	9	3426.47	2526.47	9	7.40%	3368.81	2468.81	9	5.59%
R3	1454.60	1054.60	4	1548.34	1148.34	4	6.44%	1518.80	1118.80	4	4.41%
R4	2121.94	1621.94	5	2239.94	1739.94	5	5.56%	2193.15	1693.15	5	3.36%
Avg	2283.13	1608.13	6.8	2491.95	1816.95	6.8	9.15%	2426.41	1751.41	6.8	6.28%

is located within the area has lower total costs at 1884.55 (Avg) and 1872.24 (Best), compared to the costs of 2982.28 (Avg) and 2965.02 (Best) of instance R2 with outside depot. Moreover, the R1 instance has more requests and options, which means a potentially higher travel cost due to the increased service, but the R2 instance incurs significantly higher travel cost from the depot to the area, and the results indicate that this cost is substantial. Another interesting observation is that sparser location areas lead to increased SDL usage and higher utilization rates of SDL capacity. Instances like R3 and R4, which have the same numbers of SDL, show higher average and best case utilization rates of SDL capacity for the more sparsely distributed R3 instance (75.81% and 75.58%, respectively), compared to R4 instance (71.59% for both cases). This pattern is also evident between R1 and R2 instances, where R1 instance uses one more SDL in the best case than R2 instance. The reason behind this observation is the cost-effectiveness consideration: visiting more IDL in sparse areas increases travel cost. Conversely, the visiting for SDL can be a potential detour compared to nearby delivering to IDL when the delivery area is dense.

6.3. Analysis on the construction of initial feasible solution

In the real world, obtaining a feasible solution quickly is crucial, especially in sudden situations such as temporary customer order cancellations. Therefore, it is interesting to evaluate the performance of the proposed RFIH method on real-world instances. Since the works of Tilk et al. (2021) and Dumez et al. (2021) did not provide construction methods for the initial feasible solution, the construction methods from two works of VRPDO-related problems are introduced as state-of-the-art methods and are slightly modified to apply in our VRPDO: greedy randomized adaptive search procedures (Greedy) for VRPRDL (Reyes et al., 2017) and regret insertion heuristic (Regret) for GVRPTW (Yuan et al., 2021). For simplicity, the key details of the two state-of-the-art methods are presented in Appendix D.

To account for the randomness in all the three construction methods, each method is executed 5 runs on each instance, and the average solutions are summarized in Table 8, including the total cost (Z_{tot}), travel cost (C_{rou}), and number of vehicles(N_{veh}). Furthermore, the percentage gap (Gap) between each of the state-of-the-art methods and RFIH with respect to Z_{tot} is calculated using the same method as in Section 5.3. As shown in Table 8, it is easy to observe that RFIH consistently demonstrates the best performance, which leads to the lowest average total cost of 2283.13. In contrast, Greedy method performs the worst with the highest average total cost of 2491.95. In addition, compared to designing complex insertion methods, devoting more effort to identifying appropriate options may be more effective in obtaining high-quality initial feasible solutions for VRPDO. This is particularly evident on the challenging R1 instance, where RFIH achieves the lowest average total cost of 2365.47, and results in the gap of 16.38% and 10.97% for Greedy and Regret, respectively.

Table 9
Comparative results on the NMH components.

Ins	NMH			NMH-1				NMH-2			
	Z_{avg}	Z_{bst}	T_{avg}	Z_{avg}	Z_{bst}	T_{avg}	Gap	Z_{avg}	Z_{bst}	T_{avg}	Gap
R1	1884.55	1872.24	294.5	1943.01	1920.25	230.0	2.56%	1909.77	1899.48	281.8	1.46%
R2	2982.28	2965.02	295.1	3029.71	3010.80	228.1	1.54%	3008.37	2992.55	272.0	0.93%
R3	1271.84	1263.15	285.8	1284.27	1274.33	205.2	0.89%	1277.17	1268.59	253.8	0.43%
R4	1976.52	1969.52	297.3	2002.41	1992.94	217.9	1.19%	1989.31	1980.89	271.8	0.58%
Avg	2028.80	2017.48	293.2	2064.85	2049.58	220.3	1.59%	2046.15	2035.38	269.8	0.89%

Table 10
Results on the different ρ of RFIH.

ρ	R1		R2		R3		R4		Avg	
	Z_{tot}	T	Z_{tot}	T	Z_{tot}	T	Z_{tot}	T	Z_{tot}	T
0.05	2463.00	52.3	3201.10	14.6	1484.60	6.1	2128.81	13.2	2319.88	21.6
0.1	2478.48	33.2	3208.42	9.9	1458.45	3.4	2130.08	10.4	2318.86	14.2
0.15	2452.67	15.1	3212.32	6.5	1477.31	1.6	2145.94	4.6	2322.06	7.0
0.2	2435.55	18.1	3223.21	7.8	1504.78	1.6	2167.36	4.1	2332.72	7.9

6.4. Evaluation of algorithmic components

The NMH proficiently addresses the VRPDO through the utilization of three key components, namely the VND, the column generation, and the set-partitioning model. Therefore, it is interesting to evaluate the performance of key components of NMH. Three different algorithms are considered for comparison:

- NMH: the algorithm with all the three components;
- NMH-1: the algorithm disabling both the column generation and the set-partitioning model;
- NMH-2: the algorithm disabling only the column generation.

Consistent with the NMH in the on-the-ground needs of 3PL company, the other two extra variants are executed with a maximum runtime of 300 s per run, and run 5 times per instance.

Table 9 provides the average total cost (Z_{avg}), the best total cost (Z_{bst}) and the average runtime of obtaining the best solution (T_{avg}) for the three algorithms. The percentage gap (Gap) is calculated in the same way as in Section 5.3, but applies to Z_{bst}. Additionally, the last row (Avg) records the average results of instances. It is worth noting that NMH has the best performance compared to the other algorithms, which consistently achieves the best performance across all instances. In contrast, NMH-1 obtains the worst performance among the three algorithms but has the shortest average runtime, indicating that NMH-1 is prone to get stuck in a local optimum. Comparatively, NMH-2 successfully improves this predicament by reducing the average gap with NMH from 1.59% to 0.89%. This improvement is particularly evident on R1 and R2 instances, where the gap is reduced from 2.56% to 1.46% and from 1.54% to 0.93%, respectively. Furthermore, a comparison between NMH and NMH-2 underscores the benefits of the column generation in cost reduction, leading to an average gap of 0.89%. Clearly, NMH enhances the search capability through column generation, resulting in a rich route pool that enables the entire search process to escape the local optimum effectively. Particularly in the more challenging R1 instance, the component of column generation significantly reduces the best total cost by 1.46%.

6.5. Sensitivity analysis on parameter settings

In this section, the sensitivity analysis of key parameters are conducted. First, the impact of parameter ρ is assessed on the runtime and solution quality of RFIH. Second, it is necessary to focus on the parameters β_0 and β_1, which play important roles in service level constraints. Note that each parameter setting is run 5 times, and the NMH and RFIH follow the configuration of Sections 6.2 and 6.3, respectively.

To evaluate the impact of different ρ values on RFIH, a set of values $\{0.05, 0.1, 0.15, 0.2\}$ are tested. Table 10 reports the average total cost (Z_{tot}) and runtime (T) across four instances, along with the overall average for each ρ. Then, a line plot is shown in Fig. 10 for the average runtime by different ρ, and the values can be found in Table 10. First, the runtime of RFIH increases significantly across all instances as ρ decreases from 0.15 to 0.05, with the average runtime rising from 7 to 21.6 s in average performance. Meanwhile, an observation is that as ρ increases from 0.15 to 0.2, the runtime may increase, such as the R1 line in Fig. 10. In addition, the average cost maintains a relatively low sensitivity to ρ, which further means that setting ρ a smaller value is not necessarily appropriate, as it requires significantly more runtime without a guarantee of total cost improvement.

Table 11 shows experimental results for parameters β_0 and β_1 with sets of tuning values $\{60, 70, 80, 90\}$ and $\{70, 80, 90, 100\}$, respectively, including the total cost (Z_{tot}), travel cost (C_{rou}), the number of vehicles used (N_{veh}) and the utilization rate of SDL capacity (γ). According to Table 11, either β_0 or β_1 increases, leading to a higher average total cost, especially for the most challenging R1 instance. For example, as $\beta_0 = 60\%$, increasing β_1 from 70% to 100% raises the average total cost of R1 instance

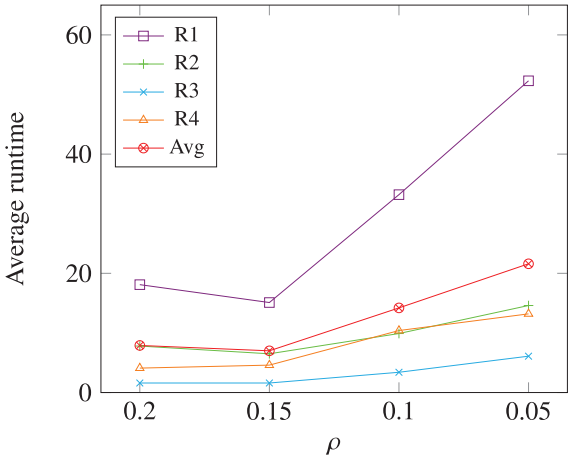


Fig. 10. Line plot for the average runtime by different ρ.
Source: Created by the authors.

Table 11
Results on the different β_0 and β_1 of NMH.

β_0	β_1	R1				R2				R3				R4			
		Z_{tot}	C_{rou}	N_{veh}	γ	Z_{tot}	C_{rou}	N_{veh}	γ	Z_{tot}	C_{rou}	N_{veh}	γ	Z_{tot}	C_{rou}	N_{veh}	γ
60%	70%	1761.44	861.44	9	82.56%	2898.19	1998.19	9	72.99%	1236.59	836.59	4	85.58%	1953.62	1453.62	5	85.91%
	80%	1771.16	871.16	9	81.56%	2903.66	2003.66	9	72.88%	1237.15	837.15	4	85.12%	1957.21	1457.21	5	85.23%
	90%	1780.71	880.71	9	81.44%	2910.56	2010.56	9	72.77%	1241.90	841.90	4	85.12%	1958.65	1458.65	5	85.00%
	100%	1828.48	928.48	9	76.89%	2978.38	2078.38	9	66.33%	1291.06	891.06	4	77.67%	1978.83	1478.83	5	80.68%
70%	80%	1865.56	965.56	9	77.89%	2926.80	2026.80	9	69.72%	1244.09	844.09	4	83.72%	1965.48	1465.48	5	82.05%
	90%	1866.99	966.99	9	77.56%	2980.08	2080.08	9	65.54%	1244.46	844.46	4	83.49%	1968.19	1468.19	5	81.36%
	100%	1958.86	1058.86	9	75.67%	2983.26	2083.26	9	62.94%	1320.96	920.96	4	77.44%	2002.72	1502.72	5	73.64%
80%	90%	1884.55	984.55	9	70.67%	2982.28	2082.28	9	64.18%	1271.84	871.84	4	75.81%	1976.52	1476.52	5	71.59%
	100%	1980.84	1080.84	9	67.22%	2988.28	2088.28	9	61.81%	1336.33	936.33	4	69.07%	2009.23	1509.23	5	70.45%
90%	100%	2083.04	1183.04	9	59.44%	3045.29	2145.29	9	55.37%	1361.26	961.26	4	62.79%	2040.96	1540.96	5	60.23%

from 1761.44 to 1828.48. In fact, the number of vehicles used is relatively insensitive to β_0 and β_1, whereas β_0 and β_1 have more obvious impact on the travel cost. Finally, as β_0 or β_1 increases, the average utilization rate of SDL capacity declines, which means that overemphasis on improving the service level may cause the inefficient utilization of SDL capacity.

7. Conclusion

In this paper, a novel neighborhood-based matheuristic algorithm is tailored specifically for the VRPDO, which efficiently explores the solution space from both primal and dual perspectives to obtain high-quality solutions. Then, a relax-and-fix based insertion heuristic is developed to obtain initial feasible solutions. In addition, a segment-based evaluation scheme is extended for the subtour verification to speed up the feasibility check of a given solution. Our experimental results show that the proposed algorithm has correctness and competitiveness on both small-scale and large-scale benchmarks through comparisons with the state-of-the-art methods proposed in the literature. Moreover, a real-world case study of VRPDO involving a third-party logistics company in China is introduced. The performance of the construction algorithm and key components proposed in our algorithm are further validated.

Although NMH demonstrates impressive performances in both state-of-the-art and real-world benchmark instances, future works can continuously improve the components of our proposed algorithm from the primal and dual perspectives. In addition, the NMH is not only proposed as a novel method to effectively address VRPDO, but can also be regarded as a general framework to solve other contexts with alternative delivery options, such as the waste collection system with door-to-door and container-bring services (Gläser and Stücken, 2021; Lavigne et al., 2023), crowdsourced last-mile delivery (Wang et al., 2023; Cheng et al., 2024; Gao et al., 2023b), and healthcare-related routing with therapist scheduling (Gartner et al., 2018; Jungwirth et al., 2022). Finally, the proposed NMH framework explores high-quality solutions by simultaneously leveraging both the primal and dual perspectives, which provides strong generalizable capabilities for the well-established and complicated VRP variants, such as VRP with time-dependent travel time and pickup-and-delivery problems, etc, and thus this method shows potential in further improving state-of-the-art solutions for these variants.

CRediT authorship contribution statement

Kai Yang: Writing – original draft, Visualization, Validation, Software, Methodology, Formal analysis, Conceptualization. **Zhenzhen Zhang:** Writing – review & editing, Validation, Supervision, Project administration, Methodology, Investigation, Funding acquisition, Conceptualization. **Jingyi Zhao:** Writing – original draft, Validation, Methodology, Formal analysis. **Zhe Liang:** Writing – review & editing, Supervision, Resources, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgment

This research was partially supported by the National Natural Science Foundation of China (Grant No. 72422012, 72101187 and 72021002) and the Fundamental Research Funds for the Central Universities.

Appendix A. Supplementary data

Supplementary material related to this article can be found online at https://doi.org/10.1016/j.tre.2025.104366.

Data availability

Data will be made available on request.

References

Archetti, C., Bouchard, M., Desaulniers, G., 2011. Enhanced branch and price and cut for vehicle routing with split deliveries and time windows. Transp. Sci. 45 (3), 285–298.
Baldacci, R., Christofides, N., Mingozzi, A., 2008. An exact algorithm for the vehicle routing problem based on the set partitioning formulation with additional cuts. Math. Program. 115, 351–385.
Beheshti, A.K., Hejazi, S.R., 2015. A novel hybrid column generation-metaheuristic approach for the vehicle routing problem with general soft time window. Inform. Sci. 316, 598–615.
Cheng, R., Fessler, A., Nielsen, O.A., Larsen, A., Jiang, Y., 2024. Assessing the potential impacts of public transport-based crowdshipping: A case study in a central district of Copenhagen. Front. Eng. Manag. 11 (4), 697–709.
Christiaens, J., Vanden Berghe, G., 2020. Slack induction by string removals for vehicle routing problems. Transp. Sci. 54 (2), 417–433.
Costa, L., Contardo, C., Desaulniers, G., 2019. Exact branch-price-and-cut algorithms for vehicle routing. Transp. Sci. 53 (4), 946–985.
Dantzig, G.B., Ramser, J.H., 1959. The truck dispatching problem. Manag. Sci. 6 (1), 80–91.
Demir, E., Bektaş, T., Laporte, G., 2012. An adaptive large neighborhood search heuristic for the pollution-routing problem. European J. Oper. Res. 223 (2), 346–359.
Desaulniers, G., Lessard, F., Hadjar, A., 2006. Tabu search, generalized k-path inequalities, and partial elementarity for the vehicle routing problem with time windows. Les Cah. Du GERAD ISSN 711, 2440.
Dumez, D., Lehuédé, F., Péton, O., 2021. A large neighborhood search approach to the vehicle routing problem with delivery options. Transp. Res. Part B: Methodol. 144, 103–132.
Frey, C.M., Jungwirth, A., Frey, M., Kolisch, R., 2023. The vehicle routing problem with time windows and flexible delivery locations. European J. Oper. Res. 308 (3), 1142–1159.
Gao, Z., Huang, H.-J., Guo, J., Yang, L., Wu, J., 2023b. Future urban transport management. Front. Eng. Manag. 10 (3), 534–539.
Gao, J., Zhen, L., Laporte, G., He, X., 2023a. Scheduling trucks and drones for cooperative deliveries. Transp. Res. Part E: Logist. Transp. Rev. 178, 103267, URL https://www.sciencedirect.com/science/article/pii/S1366554523002557.
Gartner, D., Frey, M., Kolisch, R., 2018. Hospital-wide therapist scheduling and routing: Exact and heuristic methods. IISE Trans. Heal. Syst. Eng. 8 (4), 268–279.
Gläser, S., Stücken, M., 2021. Introduction of an underground waste container system–model and solution approaches. European J. Oper. Res. 295 (2), 675–689.
Janinhoff, L., Klein, R., Scholz, D., 2024. Multitrip vehicle routing with delivery options: A data-driven application to the parcel industry. OR Spectrum 46 (2), 241–294.
Jungwirth, A., Desaulniers, G., Frey, M., Kolisch, R., 2022. Exact branch-price-and-cut for a hospital therapist scheduling problem with flexible service locations and time-dependent location capacity. INFORMS J. Comput. 34 (2), 1157–1175.
Kötschau, R., Scherr, N., Tilk, C., Ehmke, J.F., 2025. Mobile home delivery parcel lockers. Transp. Res. Part E: Logist. Transp. Rev. 193, 103867.
Lavigne, C., Inghels, D., Dullaert, W., Dewil, R., 2023. A memetic algorithm for solving rich waste collection problems. European J. Oper. Res. 308 (2), 581–604.
López-Ibáñez, M., Dubois-Lacoste, J., Cáceres, L.P., Birattari, M., Stützle, T., 2016. The irace package: Iterated racing for automatic algorithm configuration. Oper. Res. Perspect. 3, 43–58.
Najy, W., Archetti, C., Diabat, A., 2025. A column generation-based matheuristic for an inventory-routing problem with driver-route consistency. European J. Oper. Res. 324 (2), 382–397, URL https://www.sciencedirect.com/science/article/pii/S0377221725001109.
Pan, B., Zhang, Z., Lim, A., 2021a. A hybrid algorithm for time-dependent vehicle routing problem with time windows. Comput. Oper. Res. 128, 105193.
Pan, B., Zhang, Z., Lim, A., 2021b. Multi-trip time-dependent vehicle routing problem with time windows. European J. Oper. Res. 291 (1), 218–231.
Parragh, S.N., Schmid, V., 2013. Hybrid column generation and large neighborhood search for the dial-a-ride problem. Comput. Oper. Res. 40 (1), 490–497.
Reyes, D., Savelsbergh, M., Toriello, A., 2017. Vehicle routing with roaming delivery locations. Transp. Res. Part C: Emerg. Technol. 80, 71–91.
Ropke, S., Pisinger, D., 2006. A unified heuristic for a large class of vehicle routing problems with backhauls. European J. Oper. Res. 171 (3), 750–775.
Tilk, C., Olkis, K., Irnich, S., 2021. The last-mile vehicle routing problem with delivery options. OR Spectrum 43 (4), 877–904.

Vidal, T., Crainic, T.G., Gendreau, M., Prins, C., 2013. A hybrid genetic algorithm with adaptive diversity management for a large class of vehicle routing problems with time-windows. Comput. Oper. Res. 40 (1), 475–489.

Wang, L., Xu, M., Qin, H., 2023. Joint optimization of parcel allocation and crowd routing for crowdsourced last-mile delivery. Transp. Res. Part B: Methodol. 171, 111–135, URL https://www.sciencedirect.com/science/article/pii/S0191261523000504.

Wang, M., Zhang, C., Bell, M.G., Miao, L., 2022. A branch-and-price algorithm for location-routing problems with pick-up stations in the last-mile distribution system. European J. Oper. Res. 303 (3), 1258–1276.

Wei, L., Zhang, Z., Lim, A., 2014. An adaptive variable neighborhood search for a heterogeneous fleet vehicle routing problem with three-dimensional loading constraints. IEEE Comput. Intell. Mag. 9 (4), 18–30.

Yu, V.F., Anh, P.T., 2025. The electric vehicle routing problem with time windows, partial recharges, and covering locations. Int. Trans. Oper. Res. 1–39, arXiv:https://onlinelibrary.wiley.com/doi/pdf/10.1111/itor.70018. URL https://onlinelibrary.wiley.com/doi/abs/10.1111/itor.70018. https://onlinelibrary.wiley.com/doi/abs/10.1111/itor.70018. Online, (Accessed 26 February).

Yuan, Y., Cattaruzza, D., Ogier, M., Semet, F., Vigo, D., 2021. A column generation based heuristic for the generalized vehicle routing problem with time windows. Transp. Res. Part E: Logist. Transp. Rev. 152, 102391.

Zhang, Z., Che, Y., Liang, Z., 2024. Split-demand multi-trip vehicle routing problem with simultaneous pickup and delivery in airport baggage transit. European J. Oper. Res. 312 (3), 996–1010.

Zhang, Z., Luo, Z., Qin, H., Lim, A., 2019. Exact algorithms for the vehicle routing problem with time windows and combinatorial auction. Transp. Sci. 53 (2), 427–441.